接口类量产问题
排查指南

版本号: 2.7

发布日期: 2025.06.24

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
0.5	2018.12.05	AW1361	文档初始版本
1.0	2019.03.01	AW1361	修订为 V1.0 版本
1.1	2019.04.11	AW1458	补充完善 USB 调试手段
1.2	2019.05.29	AW1430	完善 TWI 调试手段，增加案例 1.4.3
1.3	2019.06.18	AW1549	增加 uart 案例
1.4	2019.06.20	AW1458	增加同步传输调试手段
1.5	2019.12.30	AW1637	完善 TWI 调试手段，增加案例 1.4.4~1.4.6
1.6	2019.12.30	AW1636	增加案例 2.4.4、6.3
1.7	2020.03.03	AW1458	增加案例 3.3.10、3.3.11
1.8	2020.05.19	AW1636	增加案例 6.4
1.9	2020.06.08	AW1636	增加案例 2.4.5、2.4.6
2.0	2020.09.15	AW1636	增加案例 1.4.7、6.5
2.1	2020.10.27	AW1636	增加 SMART CARD 模块
2.2	2020.12.10	XAA0191	增加案例 7.6 和 7.7
2.3	2020.12.10	AW1736	修改 USB 常见问题排查章节，将一些内容链接到《Linux_USB_ 开发指南》中统一维护。
2.4	2023.11.23	AW1881	完善文档格式，新增 A523 相关寄存器信息。
2.5	2024.11.1	AWA2215	新增 V821 相关寄存器信息。
2.6	2025.04.03	AWA2215	新增 A733/T736 相关寄存器信息。
2.7	2025.06.24	AWA2215	新增 MR153/T153/MR536/T536 相关寄存器信息。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 1
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1

2 TWI 2
2.1 日志信息调试 2
2.2 基础调试手段 3
2.2.1 TWI 控制器基地址 3
2.2.2 TWI 寄存器调试 4
2.2.2.1 读写单个寄存器 5
2.2.2.2 确认 PIN function 5
2.2.2.3 确认 CLK source 和 CLK gating 7
2.2.2.4 确认中断号 8
2.2.2.5 出错代码对照 10
2.3 基础配置检查 11
2.4 常见问题排查 13
2.4.1 START can’t sendout! 13
2.4.1.1 问题现象 13
2.4.1.2 排查步骤 14
2.4.2 STOP can’t sendout! 15
2.4.2.1 问题现象 15
2.4.2.2 问题分析： 15
2.4.3 incomplete xfer (status: 0x20… 16
2.4.3.1 问题现象 16
2.4.3.2 排查步骤 16
2.4.3.3 排查步骤： 16
2.4.4 incomplete xfer (status: 0x48… 17
2.4.4.1 问题现象 17
2.4.4.2 问题分析 17
2.4.4.3 排查步骤 17
2.4.5 xfer timeout 18
2.4.5.1 问题现象 18
2.4.5.2 问题分析 18
2.4.5.3 排查步骤 18
2.4.6 i2c state isn’t idle 19
2.4.6.1 问题现象 19
2.4.6.2 问题分析 20

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.4.6.3 排查步骤 20
2.4.7 i2c 和 pmu 通信失败导致不能关机 20
2.4.7.1 问题现象 20
2.4.7.2 问题分析 21
2.4.7.3 排查步骤： 21
2.4.8 i2c 作为从模式时，master 通信失败 21
2.4.8.1 问题现象 21
2.4.8.2 问题分析 22
2.4.8.3 排查步骤： 22

3 SPI 23
3.1 日志信息调试 23
3.2 基础调试手段 24
3.2.1 SPI 控制器基地址 24
3.2.2 SPI 寄存器调试 25
3.3 基础配置检查 27
3.4 SPI 常见问题排查 28
3.4.1 SPI 初始化失败 28
3.4.1.1 问题现象 28
3.4.1.2 原因分析 29
3.4.1.3 排查步骤 29
3.4.2 SPI Norflash 兼容性问题 29
3.4.2.1 问题现象 29
3.4.2.2 原因分析 30
3.4.2.3 排查步骤 30
3.4.3 SPI 数据传输异常 30
3.4.3.1 问题现象 30
3.4.3.2 排查步骤 30
3.4.4 SPI 数据传输首字节丢失 32
3.4.4.1 问题现象 32
3.4.4.2 排查步骤 32
3.4.5 SPI 传输超时无响应 32
3.4.5.1 问题现象 32
3.4.5.2 排查步骤 32
3.4.6 SPI 传输 CS 提前被拉低 33
3.4.6.1 问题现象 33
3.4.6.2 排查步骤 33

4 USB 35
4.1 日志信息调试 35
4.2 基础调试手段 37
4.2.1 USB 控制器基地址 37
4.2.2 高速眼图测试命令 38

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.2.1 USB Device 眼图测试 38
4.2.2.2 USB Host 眼图测试 38
4.2.3 USB Host 驱动能力调节 39
4.2.4 USB 寄存器打印 39
4.2.4.1 USB Device 寄存器打印 40
4.2.4.2 USB Host 寄存器打印 40
4.3 基础配置检查 40
4.4 USB 常见问题排查 40
4.4.1 USB 硬件完备排查 40
4.4.1.1 USB Device 硬件完备排查 40
4.4.1.2 USB Host 硬件完备排查 42
4.4.2 USB 基本功能异常排查 44
4.4.2.1 USB Device 基本功能异常 44
4.4.2.2 USB Host 基本功能异常 46
4.4.2.3 USB 同步传输速率说明 48

5 IR-RX 49
5.1 日志信息调试 49
5.2 基础调试手段 50
5.3 基础配置检查 51
5.4 IR-RX 常见问题排查 53
5.4.1 android 平台操作红外失效排查手段 53
5.4.1.1 问题现象 53
5.4.1.2 排查步骤 53

6 LRADC（keyboard） 56
6.1 日志信息调试 56
6.2 基础调试手段 56
6.3 基础配置检查 57
6.4 LRADC 常见问题排查 58
6.4.1 按键无反应 58
6.4.1.1 问题现象 58
6.4.1.2 排查步骤 58

7 UART 60
7.1 串口打印正常，卡打印有问题 60
7.2 板上的某个串口异常 60
7.2.1 问题现象 60
7.2.2 排查步骤 60
7.3 串口上电第一次传输收到的数据是 0x00 60
7.3.1 问题现象 60
7.3.2 排查步骤 60
7.4 开启 S_UART 导致开机死机 61
7.4.1 问题现象 61

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7.4.2 排查步骤 61
7.5 UART 无法正常发送数据 62
7.5.1 问题现象 62
7.5.2 排查步骤 62
7.6 UART 串口接收数据异常 63
7.6.1 问题现象 63
7.6.2 排查步骤 63
7.6.3 原因描述 63
7.7 UART 配置波特率与时钟不匹配 63
7.7.1 问题现象 63
7.7.2 排查步骤 64

8 SMART CARD 67
8.1 SCR 无法获取 ATR 数据 67
8.1.1 问题现象 67
8.1.2 排查步骤 67

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　PIN 基地址 5
图 2-2　PB 地址偏移 5
图 2-3　PIN 功能号 6
图 2-4　打开 SUSPEND 级别调试 6
图 2-5　CLK source 7
图 2-6　CCU 基地址 7
图 2-7　TWI0 的 clk gating 偏移 8
图 2-8　TWI0 的 RST 在寄存器中的偏移量 8
图 2-9　TWI0 的 GATING 在寄存器中的偏移量 8
图 2-10　TWI 中断号 9
图 2-11　老版控制器 TWI ENGINE 的出错代码定义 10
图 2-12　新版控制器 TWI DRIVER 出错代码定义 11
图 2-13　屏幕和 I2C 通信失败日志 20
图 2-14　从模式通信失败波形 22
图 3-1　SPI 基地址 25
图 3-2　PA 偏移地址 26
图 3-3　GPIO 文档要求 26
图 3-4　ID Definitions 30
图 3-5　添加新的 flash 30
图 3-6　nor 测试用例裸测 mtdblock 分区-未开打印 31
图 3-7　nor 测试用例裸测 mtdblock 分区-开启打印 31
图 3-8　SPI cs 被提前拉低 100us 33
图 4-1　将设备配置成全速模式 45
图 5-1　查看红外接收设备 49
图 5-2　打开设备 50
图 5-3　hexdump 日志 50
图 5-4　查看红外接收的中断次数 1 50
图 5-5　查看红外接收的中断次数 2 51
图 6-1　查看 sunxi-keyboard 名称的 input 设备 56
图 6-2　打开设备 57
图 6-3　LRADC hexdump 数据 57
图 6-4　电路原理图 58

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表　格

表 1-1　适用产品列表 1
表 2-1　全志 SoC CPUX 域的 TWI 控制器基地址 3
表 2-2　续表 _ 全志 SoC CPUX 域的 TWI 控制器基地址 4
表 2-3　全志 SoC CPUS 域的 TWI 控制器基地址。 4
表 3-1　全志 SoC 的 SPI 控制器基地址 24
表 3-2　全志 SoC CPUS 域的 SPI 控制器基地址。 25
表 4-1　各个 SoC 型号对应的 USB 控制器基地址寄存器表。 37
表 5-1　全志 SoC 的 IR-RX 控制器基地址。 51

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

- 本文档提供 TWI、SPI、USB、UART 接口等常规量产问题快速排查指导。
- 本文档提供的快速排查指南工程思路如下：

常规问题场景 + 基础硬件排查 + 基础软件配置排查 + 软件快速调试排查。

- 本文档仅用于帮助开发人员快速排查和解决初级问题问题，如果按照文档中的指导仍无法解决问题，请联系全志原厂 FAE 进一步协助解决，并提供说明是否按照文档中的排查方案进行过必要的排查，提供排查过程的必要记录和数据。

1.2 目标读者

本文档主要适用于以下工程师:

- 对 TWI、SPI、USB、UART 等接口技术“零” 基础的 RD 研发或 FAE 工程人员。
- 对 TWI、SPI、USB、UART 等接口技术初级基础的 RD 研发或 FAE 工程人员。

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本
所有产品	所有版本

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 TWI

2.1 日志信息调试

- 步骤 1

设备通过 UART 串口线连接调试控制台，或使用 usb 连接使用 adb shell 进行调试。

- 步骤 2

开启设备后，查看控制台打印 Log 的信息，若有以下类似 init name: twi* 的信息（电源警告不影响），表示总线驱动注册已成功。

```
[   1.207133] sunxi_i2c_probe()2271 - [i2c0] twi_drv_used = 0
[   1.210949] sunxi_i2c_probe()2275 - [i2c0] twi_pkt_interval = 0
[   1.216824] sunxi_i2c_probe()2280 - [i2c0] no_suspend = 0
[   1.222226] twi0 supply twi not found, using dummy regulator
[   1.228213] twi_request_gpio()53 - [i2c0] init name: twi0
```

说明

若出现错误信息，或没有出现需要的第几号总线，请参考 2.3 TWI 基础配置检查进行配置。

- 步骤 3

启动后，调试时只需要打开 DEBUG_INFO 项的打印，可以在控制台输入以下命令。

```
echo 4 > /sys/module/i2c_sunxi/parameters/debug
```

说明

1. 日志打印等级的定义如下：

```
enum {
DEBUG_INIT = 1U<<0, //初始化调试打印
DEBUG_SUSPEND = 1U<<1, //休眠唤醒函数调试打印
DEBUG_INFO = 1U<<2, //传输粗略调试打印
DEBUG_INFO1 = 1U<<3, //传输全面调试打印
DEBUG_INFO2 = 1U<<4, //寄存器设置调试打印
};
```

2. 总线驱动的打印信息都带有前缀 [i2cx]，如需要查找 twi0 的打印信息，可以搜索 i2c0。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2 基础调试手段

2.2.1 TWI 控制器基地址

全志各 SoC 型号对应的 TWI 控制器基地址寄存器如下表：

表 2-1: 全志 SoC CPUX 域的 TWI 控制器基地址

TWI	TWI0	TWI1	TWI2	TWI3	TWI4	TWI5
T3	0x01c2ac00	0x01c2b000	0x01c2b400	0x01c2b800	0x01c2c000	
V5	0x05002000	0x05002400	0x05002800	0x05002c00	\	
A50	0x05002000	0x05002400	0x05002800	\		
V5V200	0x05002000	0x05002400	0x05002800	0x05002c00	\	
T7	0x05002000	0x05002400	0x05002800	0x05002c00	0x05003000	
R328	0x05002000	0x05002400	\			
A64	0x01c2ac00	0x01c2b000	0x01c2b400	\		
A63	0x05002000	0x05002400	0x05002800	0x05002c00	0x05003000	
H6	0x05002000	0x05002400	0x05002800	0x05002c00	\	
R100	0x05002000	0x05002400	0x05002800	\		
H3 pro	0x05002000	0x05002400	0x05002800	0x05002c00	\	
V459	0x05002000	0x0500400	0x05002800	0x05002c00	\	
R329	0x02502000	0x02502400	\			
A100	0x05002000	0x05002400	0x05002800	0x05002c00	0x05003000	0x05003400
A523	0x02502000	0x02502400	0x02502800	0x02502c00	0x02503000	0x02503400
V821	0x42502000	0x42502400	0x42502800	\		
A733	0x02510000	0x02511000	0x02512000	0x02513000	0x02514000	0x02515000
T736	0x02510000	0x02511000	0x02512000	0x02513000	0x02514000	0x02515000
MR153	0x02510000	0x02511000	0x02512000	0x02513000	0x02514000	0x02515000
T153	0x02510000	0x02511000	0x02512000	0x02513000	0x02514000	0x02515000
MR536	0x02510000	0x02511000	0x02512000	0x02513000	0x02514000	0x02515000

版权所有 © 珠海全志科技股份有限公司。保留一切权利

T536	0x02510000	0x02511000	0x02512000	0x02513000	0x02514000	0x02515000

表 2-2: 续表 _ 全志 SoC CPUX 域的 TWI 控制器基地址

TWI	TWI6	TWI7	TWI8	TWI9	TWI10	TWI11	TWI12
A733	0x02516000	0x02517000	0x02518000	0x02519000	0x0251a000	0x0251b000	0x0251c000
T736	0x02516000	0x02517000	0x02518000	0x02519000	0x0251a000	0x0251b000	0x0251c000
MR536	0x02516000	\					
T536	0x02516000	\					

表 2-3: 全志 SoC CPUS 域的 TWI 控制器基地址。

TWI	S_TWI0	S_TWI1	S_TWI2
H3Pro	0x07081400	\	\
V459	0x07081400	\	\
R329	0x07081400	\	\
A100	0x07081400	0x07081800	\
A523	0x07081400	0x07081800	0x07081c00
A733	0x07083000	0x07084000	0x07085000
T736	0x07083000	0x07084000	0x07085000
MR536	0x07083000	0x07084000	\
T536	0x07083000	0x07084000	\

2.2.2 TWI 寄存器调试

设备通过 UART 串口调试控制台或 adb shell 调试终端输入以下命令。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.2.1 读写单个寄存器

确认寄存器正常可操作: 以 T3 的 TWI1 为例，向 0x01c2b000 地址写入 0x00000003, 再读出进行对比，检查是否异常。

```
echo 0x01c2b000 0x00000003 > sys/class/sunxi_dump/write
cat sys/class/sunxi_dump/write
reg to_write after_write
0x01c2b000 0x00000003 0x00000003
```

⚠ 注意

如写入寄存器某些位属性为'' 写 1 清零（W1C）'' 的，写入与读出的值是会不一样的，做此实验时应忽略 W1C 位。

2.2.2.2 确认 PIN function

- 步骤 1

直接看寄存器确认 PIN 等硬件资源配置是否正确。需要查看芯片手册中关于 port 或 gpio 的文档，找到对应 PIN 的基地址与偏移，TWI 引脚功能号等。如 T3 的 twi0 对应的管脚为 PB0 和 PB1。

基地址：

Module Name	Base Address
PIO	0x01C20800

图 2-1: PIN 基地址

PB 地址偏移：

1.4.10. PB Configure Register 0

Offset: 0x24	Register Name: PB_CFG0 Default Value: 0x7777_7777

图 2-2: PB 地址偏移

- 步骤 2

读出 0x01c20824 的值：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
echo 0x01c20824 > sys/class/sunxi_dump/dump
cat sys/class/sunxi_dump/dump
0x77127322
```

- 步骤 3

找到 PB0、PB1 设 TWI 的功能号：

如下图所示，设 TWI0_SDA、TWI0_SCK 功能的 PIN 功能号都为 2，与上面读得的 bit6:4 与 bit2:0 都为 2 是一致的，这表示 PIN 的配置是正确的。

7	/	/	/	
6:4	R/W	0x7	PB1_SELECT 000: Input 010: TWI0_SDA 100: Reserved 110: Reserved	001: Output 011: Reserved 101: Reserved 111: IO Disable
3	/	/	/	
2:0	R/W	0x7	PB0_SELECT 000: Input 010: TWI0_SCK 100: Reserved 110: Reserved	001: Output 011: PLL_LOCK_DBG 101: Reserved 111: IO Disable

图 2-3: PIN 功能号

⚠ 注意

TWI 驱动引用了 runtime_suspend（此更新只针对 linux-4.4/linux-4.9，linux-3.10 没有 runtime_suspend）。在执行完一次传输后 5s 内若未再有传输，则会自动休眠，此休眠会将 PIN 功能设为 IO Disable，并且关闭时钟。打开 SUSPEND 级别调试（echo 2 > /sys/module/i2c_sunxi/parameters/debug），可以在串口看到 suspend/resume 打印，如下图所示。所以做 PIN/CLK 等基础资源确认实验的时候，需要在通讯后未休眠的时间内做完。

```
# echo 2 > /sys/module/i2c_sunxi/parameters/debug                    打开SUSPEND级别调试
# i2cget -y 1 0x50 0x01                                              读传输
[   84.847853] sun50iw6p1-pinctrl pio: request pin 229 (PH5) for twi1
[   84.856119] sun50iw6p1-pinctrl pio: request pin 230 (PH6) for twi1
[   84.863065] sunxi_i2c_runtime_resume()2405 - [i2c1] finish         resume
0x59
# [   89.893560] sun50iw6p1-pinctrl pio: request pin 229 (PH5) for twi1
[   89.900498] sun50iw6p1-pinctrl pio: request pin 230 (PH6) for twi1
[   89.907443] sunxi_i2c_runtime_suspend()2378 - [i2c1] finish        5s后suspend
```

图 2-4: 打开 SUSPEND 级别调试

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

若发现实际寄存器的 function 与 TWIn_SCK/TWIn_SDA 应设的 function 不一致:

- **步骤 1**

在对应板型的 sys_config.fex 文件中搜索 TWI 使用的 PIN 脚，如上面的例子，PB0/PB1，搜索 PB0，再搜索 PB1，看看是否有其他模块复用此 PIN。

- **步骤 2**

如果按照步骤 1 没找到，则到对应的 board.dts 搜索。

- **步骤 3**

如果按照步骤 2 也没找到，再找对应的 xxx-pinctrl.dtsi 搜索。

- **步骤 4**

1. **若按照上述步骤发现有复用，则确认此模块是否必须。非必需则关掉复用的模块再测试；必须则通过我司 FAE 跟 BSP 负责人确认，应该舍掉哪一个模块，或者寻求其他方法解决引脚复用问题。**
2. **若没有搜索到模块复用引脚，则通过我司 FAE 向 TWI 负责人或 PIN 相关同事寻求调试手段。**

2.2.2.3 确认 CLK source 和 CLK gating

- CLK source 相关配置可以在相应版本的内核目录中的 **xxx-clk.dtsi** 搜索找到, 或 bsp/include/dt-binding/clock/目录中的 **xxx-ccu.h** 搜索找到。

说明

一般而言，CPUX 域的 TWI，CLK 源是 APB2；对于 CPUS 域的 TWI，CLK 源是 APBS。

Table 1-3. TWI Clock Sources

Clock Sources	Description
APBS Bus	R-TWI in CPUS,for details on APBS refer to ***PRCM***
APB2 Bus	TWI in CPUX,for details on APB2 refer to ***CCU***

图 2-5: CLK source

- CLK gating 相关配置可以直接看寄存器确认是否正确。

对于 CPUX 域的 TWI，需要查看芯片手册中关于 ccu 的文档；对于 CPUS 域的 TWI，需要查看芯片手册里面关于 prcm 的文档。如 A733 的 TWI0 对应的 clk gating 寄存器偏移为 0x0E80。

1. CCU 基地址

Module Name	Base Address
CCU	0x02002000

图 2-6: CCU 基地址

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. TWI0 的 clk gating 偏移

Offset: 0x0E80	Register Name: TWI0_BGR_REG

图 2-7: TWI0 的 clk gating 偏移

3. 计算 TWI0 对应的 clk gating 寄存器地址为 0x02002E80
4. 读出 0x02002E80 的值

```
cd /sys/class/sunxi_dump/
echo 0x02002E80 > dump
cat dump
0x00010001
```

如下图所示，TWI0 的 RST 和 GATING 在寄存器中的偏移量分别是第 16 位和第 0 位，与上面读到的数据一致，则表示 CLK 的配置正确。

16	R/W	0x0	TWI0_RST. TWI0 Reset. 0: Assert 1: De-assert
15:5	/	/	/

图 2-8: TWI0 的 RST 在寄存器中的偏移量

0	R/W	0x0	TWI0_GATING Gating Clock For TWI0 0: Mask 1: Pass

图 2-9: TWI0 的 GATING 在寄存器中的偏移量

⚠ 注意

如上一节所示，TWI 驱动引用了 runtime suspend 的机制，解决方案参考上节。

2.2.2.4 确认中断号

中断号的确认可以查看芯片手册文档中的 GIC 文档。

- 步骤 1

版权所有 © 珠海全志科技股份有限公司。保留一切权利

通过下面的命令查看每个 TWI 对应的中断号：

```
cat /proc/interrupt
```

以 A100 的 TWI 为例，如下所示。

```
cat /proc/interrupts
CPU0 CPU1 CPU2 CPU3
350: 4 0 0 0 wakeupgen 7 Level twi0
351: 0 0 0 0 wakeupgen 8 Level twi1
352: 0 0 0 0 wakeupgen 9 Level twi2
353: 0 0 0 0 wakeupgen 10 Level twi3
354: 47332 0 0 0 wakeupgen 113 Level twi6
355: 0 0 0 0 wakeupgen 114 Level twi7
```

可以看到 twi0 对应的中断号为 7。

注意

在 LINUX 内核中，GIC 中断的编号需要在此基础上加上 32。

- 步骤 2 查看芯片手册文档上的中断号：

TWI0	39	0x9C		
TWI1	40	0xA0		
TWI2	41	0xA4		
TWI3	42	0xA8		
TWI4	43	0xAC		
TWI5	44	0xB0		

图 2-10: TWI 中断号

可以看到 TWI0 的中断号无误。

说明

关于 TWI 的中断号配置：首先，到板级目录的 sys_config.fex 查看；如若没有，到相同目录下的 board.dts 查看；最后到内核下的 xxx.dtsi 查看。

注意

在设备树中，中断号需要在芯片手册文档中的中断号的基础上减去 32。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.2.5 出错代码对照

TWI ENGIEN（sun8iw16 之前的平台都只有 TWI ENGIEN）出错代码对照：

1.5.5. TWI Status Register(Default Value:0x0000_00F8)

Offset: 0x0010			Register Name: TWI_STAT
Bit	Read/Write	Default/Hex	Description
31:8	/	/	/
7:0	R	0xF8	STA Status Information Byte **Code Status** 0x00: Bus error 0x08: START condition transmitted 0x10: Repeated START condition transmitted 0x18: Address + Write bit transmitted, ACK received 0x20: Address + Write bit transmitted, ACK not received 0x28: Data byte transmitted in master mode, ACK received 0x30: Data byte transmitted in master mode, ACK not received 0x38: Arbitration lost in address or data byte 0x40: Address + Read bit transmitted, ACK received 0x48: Address + Read bit transmitted, ACK not received 0x50: Data byte received in master mode, ACK transmitted 0x58: Data byte received in master mode, not ACK transmitted 0x60: Slave address + Write bit received, ACK transmitted 0x68: Arbitration lost in address as master, slave address + Write bit received, ACK transmitted 0x70: General Call address received, ACK transmitted 0x78: Arbitration lost in address as master, General Call address received, ACK transmitted 0x80: Data byte received after slave address received, ACK transmitted 0x88: Data byte received after slave address received, not ACK transmitted 0x90: Data byte received after General Call received, ACK transmitted 0x98: Data byte received after General Call received, not ACK transmitted 0xA0: STOP or repeated START condition received in slave mode

图 2-11: 老版控制器 TWI ENGINE 的出错代码定义

TWI DRIVER（sun8iw16 之后的平台包含 TWI ENGIEN 与 TWI DRIVER）出错代码对照：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

23:16	R	0xf8	TWI_STA 0x00: bus error 0x08: START condition transmitted 0x10: RepeatedSTART condition transmitted 0x18: Address+Write bit transmitted, ACK received 0x20: Address+Write bit transmitted, ACK not received 0x28: Data byte transmitted in master mode, ACK received 0x30: Data byte transmitted in master mode, ACK not received 0x38: Arbitration lost in address or data byte 0x40: Address+Read bit transmitted, ACK received 0x48: Address+Read bit transmitted, ACK not received 0x50: Data byte received in master mode, ACK received 0x58: Data byte received in master mode, ACK not received 0x01: Timeout when sending 9th SCL clk Other: Reserved

图 2-12: 新版控制器 TWI DRIVER 出错代码定义

2.3 基础配置检查

- 步骤 1

TWI device tree 配置如下，需要检查和注意的是**基地址、中断号和频率**。请查看芯片芯片手册确认设置，TWI0 频率最高 400KHZ，TWI1~TWIn 频率最高 200KHZ，TWI1~TWIn 一般设 100KHZ，**{中断号 = [芯片手册的中断号]-32}**。

```
twi0:twi@0x01c2ac00{
  compatible = "allwinner,sun50i-twi";
  device_type = "twi0";
  reg = <0x0 0x01c2ac00 0x0 0x400>;
  interrupts = <GIC_SPI 6 IRQ_TYPE_LEVEL_HIGH>;
  clocks = <&clk_twi0>;
  clock-frequency = <400000>;
  pinctrl-names = "default";
  pinctrl-0 = <&twi0_pins_a>;
  status = "okay";
};
```

- 步骤 2

board.dts 包含使能，引脚和控制方式配置。

```
twi0_pins_a: twi0@0 {
   allwinner,pins = "PH0", "PH1";
   allwinner,pname = "twi0_scl", "twi0_sda";
   allwinner,function = "twi0";
   allwinner,muxsel = <2>;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    allwinner,drive = <1>;
    allwinner,pull = <0>;
};

twi0_pins_b: twi0@1 {
    allwinner,pins = "PH0", "PH1";
    allwinner,function = "io_disabled";
    allwinner,muxsel = <7>;
    allwinner,drive = <1>;
    allwinner,pull = <0>;
};

twi0: twi@0x05002000{
    clock-frequency = <400000>;
    pinctrl-0 = <&twi0_pins_a>;
    pinctrl-1 = <&twi0_pins_b>;
    twi-supply = <&reg_ldoio0>;
    status = "okay";
};
```

Linux-5.10 及以上内核版本 board.dts 配置：

```
twi0_pins_default: twi0@0 {
    pins = "PH0", "PH1";
    function = "twi0";
    drive-strength = <10>;
    bias-pull-up;
};

twi0_pins_sleep: twi0@1 {
    pins = "PH0", "PH1";
    function = "io_disabled";
};

&twi0 {
    clock-frequency = <400000>;
    pinctrl-0 = <&twi0_pins_default>;
    pinctrl-1 = <&twi0_pins_sleep>;
    pinctrl-names = "default", "sleep";
    /* For stability and backwards compatibility, we recommend setting  'twi_drv_used'  to 1 */
    twi_drv_used = <1>;
    twi-supply = <&reg_dcdc1>;
    status = "okay";
};
```

- 步骤 3

sys_config.fex 包含使能，引脚和控制方式配置。

说明

Linux-5.10 及以上内核版本 twi 不再使用 sys_config.fex 配置

```
[twi0]
twi0_used     = 1
twi0_scl      = port:PH13<5><default><default><default>
twi0_sda      = port:PH14<5><default><default><default>
twi_drv_used      = 1
twi_regulator    =  "vcc-io"
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[twi0_suspend]
twi0_scl      = port:PH13<7><default><default><default>
twi0_sda      = port:PH14<7><default><default><default>
```

其中：

1. twi0_used 置为 1 表示使能 TWI0 控制器，0 表示不使能；
2. twi0_scl 和 twi0_sda 用于配置相应的 GPIO，如上使用 PH13 和 PH14 的管脚功能 <5> 为 twi0。
3. twi0_suspend 是休眠时 PIN 脚的状态. 如果此项没配，而 DTS 的 PIN 配置和 SYSCONFIG 的配置不是同一套 PIN，则休眠的时候会报 PIN 冲突，所以使用时需要检查此项有没有配好；
4. twi_drv_used 是新 TWI 控制器 TWI DRIVER 和老一套控制器 TWI ENGINE 的开关，1 为 TWI DRIVER，0 为 TWI ENGINE；
5. twi_regulator（twi-supply）是 SDA 和 SCL 引脚的上拉电配置。TWI 引脚内部驱动能力较弱，所以需要上拉才能发出信号，否则通讯时控制器会报 START can’t send out，此项配置需要查看原理图，确认 TWI 引脚的电是哪一路，并配上相应那路电的名字（如果电源由外接上拉电阻提供，不需要配置）。

- 步骤 4

menuconfig 的配置。

在命令行中进入内核 linux 目录，执行 make ARCH=arm menuconfig（64 位平台执行 make ARCH=arm64 menuconfig）进入配置主界面，并按以下步骤操作：

```
Device Drivers --->
 |----<*>I2C support --->
 |   |----<*>I2C devive interface
 |   |----I2C Hardware Bus support --->
 |   |   |----<*>SUNXI I2C controller
```

说明

为了确保选对 menuconfig，需要在 linux 目录下检查.config 文件，如 linux-4.9 文件夹下的.config 文件中查看 CONFIG_I2C、CONFIG_I2C_CHARDEV、CONFIG_I2C_SUNXI 的值是否都为 y。

2.4 常见问题排查

2.4.1 START can’t sendout!

2.4.1.1 问题现象

log 信息提示：START can’t sendout!

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
sunxi_i2c_do_xfer()1865 - [i2c1] START can't sendout!
```

2.4.1.2 排查步骤

- 步骤 1

根据原理图，查看 TWI-SCK 和 TWI-SDA 是否经过合适的上拉电阻接到 3.3v 电压。

- 步骤 2

用万用表量 SDA 与 SCL 初始电压，看电压是否在 3.3V 附近（断开此 TWI 控制器所有外设硬件连接与软件通讯进程）。

- 步骤 3

核查引脚配置以及 clk 配置是否如 1.3 的配置进行正确设置。

- 步骤 4

测试 PIN 的功能是否正常。

1. 利用寄存器读写的方式，将 PIN 功能直接设为 INPUT 功能。

```
echo [reg] [val] > /sys/class/sunxi_dump/write
```

2. 将 PIN 上拉和接地改变 PIN 状态，读 PIN 的状态，看是否匹配。

```
echo [reg,reg] > /sys/class/sunxi_dump/dump;cat dump
```

- 步骤 5

测试 CLK 的功能是否正常。

1. 利用寄存器读写的方式，将 TWI 的 CLK gating 等打开。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
echo [reg] [val] > /sys/class/sunxi_dump/write
```

2. 读取相应 TWI 的寄存器信息，读 TWI 寄存器的数据查看寄存器数据是否正常。

```
echo [reg] ,[len]> /sys/class/sunxi_dump/dump
```

- 步骤 6

通过我司 FAE 找 TWI 软件负责人以及硬件负责人调试。

2.4.2 STOP can’t sendout!

2.4.2.1 问题现象

log 信息提示：STOP can’t sendout!

```
twi_stop()511 - [i2c4] STOP can't sendout!
sunxi_i2c_core_process()1726 - [i2c4] STOP failed!
```

2.4.2.2 问题分析：

- 步骤 1

根据原理图，查看 TWI-SCK 和 TWI-SDA 是否经过合适的上拉电阻接到 3.3v 电压。

- 步骤 2

用万用表量 SDA 与 SCL 初始电压，看电压是否在 3.3V 附近（断开此 TWI 控制器所有外设硬件连接与软件通讯进程）。

- 步骤 3

测试 PIN 的功能是否正常。

1. 利用寄存器读写的方式，将 PIN 功能直接设为 INPUT 功能

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
echo [reg] [val] > /sys/class/sunxi_dump/write
```

2. 将 PIN 上拉和接地改变 PIN 状态，读 PIN 的状态, 看是否匹配。

```
echo [reg,reg] > /sys/class/sunxi_dump/dump;cat dump
```

- 步骤 4

通过我司 FAE 寻求 TWI 软件负责人以及硬件负责人协助调试。

2.4.3 incomplete xfer (status: 0x20…

2.4.3.1 问题现象

log 信息提示：STOP can’t sendout!

```
twi_stop()511 - [i2c4] STOP can't sendout!
sunxi_i2c_core_process()1726 - [i2c4] STOP failed!
```

2.4.3.2 排查步骤

此错误表示主控已经发送了 SLAVE ADDR + WRITE，但是设备没有回 ACK，表示设备无响应。应该检查**是否未接设备、接触不良、设备损坏和上电时序不正确导致的设备未就绪等问题**。

2.4.3.3 排查步骤：

- 步骤 1

更换 TWI 总线下的设备为 at24c16，用 i2ctools 读写 at24c16 看看是否成功，成功则表明总线工作正常。

- 步骤 2

排查设备是否可以正常工作以及设备与 I2C 之间的硬件接口是否完好。

- 步骤 3

版权所有 © 珠海全志科技股份有限公司。保留一切权利

详细了解当前需要操作的设备的**初始化方法，工作时序，使用方法**，排查因初始化设备不正确导致通讯失败。

- 步骤 4

通过我司 FAE 找 TWI 负责人调试。

2.4.4 incomplete xfer (status: 0x48…

2.4.4.1 问题现象

log 信息提示：STOP can’t sendout!

```
[1658.926643] sunxi_i2c_do_xfer()1936 - [i2c0] incomplete xfer (status: 0x48, dev addr: 0x50)
```

2.4.4.2 问题分析

此错误表示主控已经发送了 SLAVE ADDR + READ，但是设备没有回 ACK，表明设备无响应。应该检查**是否未接设备、接触不良、设备损坏和上电时序不正确导致的设备未就绪等问题**。

2.4.4.3 排查步骤

- 步骤 1

更换 TWI 总线下的设备为 at24c16，用 i2ctools 读写 at24c16 看看是否成功，成功则表明总线工作正常。

- 步骤 2

排查设备是否可以正常工作以及设备与 I2C 之间的硬件接口是否完好。

- 步骤 3

详细了解当前需要操作的设备的初始化方法，工作时序，使用方法，排查因初始化设备不正确导致通讯失败。

- 步骤 4

通过我司 FAE 找 TWI 负责人调试。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.4.5 xfer timeout

2.4.5.1 问题现象

log 信息提示：STOP can’t sendout!

```
[123.681219] sunxi_i2c_do_xfer()1914 - [i2c3] xfer timeout (dev addr:0x50)
```

2.4.5.2 问题分析

此错误表示主控已经发送完起始信号，但是在与设备通信的过程中无法正常完成数据发送与接收，导致最终没有发出终止信号来结束 I2C，传输超时。应该检查**引脚配置是否正常，CLK 配置是否正常，TWI 寄存器数据是否正常，是否有其他设备干扰，中断是否正常等问题。**

2.4.5.3 排查步骤

- 步骤 1

按照 1.3 节核实 TWI 资源配置是否正确。

- 步骤 2

根据原理图，查看 TWI-SCK 和 TWI-SDA 是否经过合适的上拉电阻接到 3.3v 电压。

- 步骤 3

用万用表量 SDA 与 SCL 初始电压，看电压是否在 3.3V 附近（断开此 TWI 控制器所有外设硬件连接与软件通讯进程）。

- 步骤 4

关闭其他 TWI 设备，重新进行烧录测试 TWI 功能是否正常。

- 步骤 5

测试 PIN 的功能是否正常。

1. 利用寄存器读写的方式，将 PIN 功能直接设为 INPUT 功能。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
echo [reg] [val] > /sys/class/sunxi_dump/write
```

2. 将 PIN 上拉和接地改变 PIN 状态，读 PIN 的状态，看是否匹配。

```
echo [reg,reg] > /sys/class/sunxi_dump/dump;cat dump
```

- 步骤 6

测试 CLK 的功能是否正常。

1. 利用寄存器读写的方式，将 TWI 的 CLK gating 等打开。

```
echo [reg] [val] > /sys/class/sunxi_dump/write
```

2. 读取相应 TWI 的寄存器信息，读 TWI 寄存器的数据，查看寄存器数据是否正常。

```
echo [reg] ,[len]> /sys/class/sunxi_dump/dump
```

- 步骤 7

根据相关的 LOG 跟踪 TWI 代码执行流程，分析报错原因。

- 步骤 8

通过我司 FAE 找 TWI 负责人调试。

2.4.6 i2c state isn’t idle

2.4.6.1 问题现象

log 信息提示：i2c state(0x0) isn’t idle(0xf8)

```
twi_stop()521 - [i2c1] i2c state(0xf8)
sunxi_i2c_core_process()1726 - [i2c1] STOP failed!
sunxi_i2c_do_xfer()1921 - [i2c1] incomplete xfer (status: 0xff, dev addr: 0x3c)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.4.6.2 问题分析

此错误表示 I2C 总线处于非空闲状态。应该检查**其他设备是否占用 I2C 总线，I2C 的引脚配置是否正常，I2C 的引脚电压是否正常等问题**。

2.4.6.3 排查步骤

- 步骤 1

查看是否有其他设备在占用 I2C 设备，通过设备树配置以及查看内核下目录（/sys/class/i2c-dev/i2c-x/device）查看是否存在其他地址的设备在使用 I2C 总线。

- 步骤 2

用万用表量 SDA 与 SCL 电压，检查 SDA 和 SCL 的电压是否正常，波动情况是否正常。

- 步骤 3

检查 I2C 引脚配置是否正常，如引脚驱动配置，引脚上拉配置等。

- 步骤 4

通过我司 FAE 找 TWI 负责人调试。

2.4.7 i2c 和 pmu 通信失败导致不能关机

2.4.7.1 问题现象

屏幕和 I2C 通信失败，打印寄存器 log 如下：

```
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_get_status:76: twi[0x10] status is 0x000000f9
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_dump:52: twi_reg0x00: 0x00000000
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_dump:53: twi_reg0x04: 0x00000000
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_dump:54: twi_reg0x0c: 0x00000060
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_dump:55: twi_reg0x10: 0x000000f9
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_dump:56: twi_reg0x14: 0x00000058
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_dump:57: twi_reg0x18: 0x00000000
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_dump:58: twi_reg0x1c: 0x00000000
[20200827-222257]ERROR:   TEE-CORE:sunxi_twi_dump:59: twi_reg0x20: 0x00000020
```

图 2-13: 屏幕和 I2C 通信失败日志

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.4.7.2 问题分析

twi 控制器 0x20 寄存器为 0x20，此时 scl 为高，sda 被拉低，传输状态为 0xf9，为未完成态。

2.4.7.3 排查步骤：

- 步骤 1

板子是否接有 pmu 器件。如果没接有但是设备树又开启了 pmu 节点，则会出现此问题。

- 步骤 2

传输状态为 0xf9，说明有数据通信时由于系统关机导致通信被切断了，软件设计缺陷。

- 步骤 3

可以通过在 TWI 驱动里加上 shutdown 函数，确保关机时 twi 的传输状态是 0xf8，电平都为空闲状态。不过因为 twishutdown 之后 cpus 可能还会调频调压用到 twi，所以此举会导致 shutdown 之后的 cpus 调频调压失败。

- 步骤 4

如果不能接受步骤 3 的风险点，可以在 opt 代码里面判断 twi 的状态，如果 twi 总线被设备拉住导致挂死，可以软件模拟 9 个 clock 来释放总线，之后即可正常通信。

2.4.8 i2c 作为从模式时，master 通信失败

2.4.8.1 问题现象

主机和 SoC I2C 从接口通信失败，表现为主设备端读数据异常或从设备端驱动状态异常，Log 如下：

主机端

```
[  77.718825] twi_restart()510 - [i2c0] Restart can't sendout!
[  77.725449] sunxi_i2c_core_process()1741 - [i2c0] twi_regulator:
[  77.732393] twi_stop()528 - [i2c0] STOP can't sendout!
[  77.738172] sunxi_i2c_core_process()1832 - [i2c0] STOP failed!
[  77.744752] sunxi_i2c_do_xfer()2029 - [i2c0] incomplete xfer (status: 0xfffffffd, dev addr: 0x50)
```

从机端

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[ 428.037818] sunxi_i2c_slave_core_process()1655 - [i2c1] slave address = 0x50, i2c_state = 0x5, irq_state = 0x0
[ 428.049091] sunxi_i2c_slave_core_process()1663 - [i2c1] slave 0x50 error status 0, need reset
```

2.4.8.2 问题分析

通过波形分析看到，当出错时，从机端 ACK 信号未成功发出，导致主机状态异常

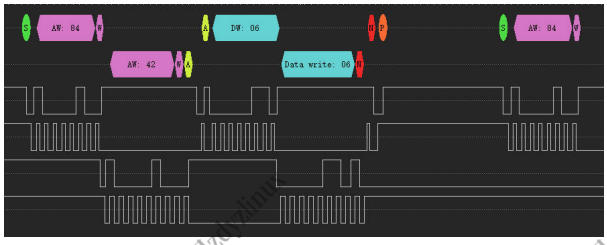

图 2-14: 从模式通信失败波形

2.4.8.3 排查步骤：

- 步骤 1

检查软硬件配置是否正确，包括 Pinmux 和上拉电阻电压等参数。均没有问题，问题仍然存在。

- 步骤 2

怀疑主机端发送时序问题，更换 master 为 USB 转 I2C 工具，仍然复现。

- 步骤 3

怀疑总线上其他器件干扰，将该总线上的其他 i2c 设备去除，只连接 SoC 作为 Slave，问题排除。

- 步骤 4

若还是有问题，通过我司 FAE 找 TWI 负责人调试。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 SPI

3.1 日志信息调试

- 步骤 1

设备通过 UART 串口线连接调试控制台，或使用 usb 连接使用 adb shell 进行调试。

- 步骤 2

开启设备后，查看控制台 log 的信息，若有以下类似 [spi3]: driver probe succeed 的信息，表示总线驱动注册已成功。

```
[   0.936569] sunxi_spi_clk_init()1599 - [spi3] mclk 100000000
[   0.942856] spi spi3: master is unqueued, this is deprecated
[   0.948760] sunxi_spi_probe()1956 - [spi3]: driver probe succeed, base f085a000, irq 62
```

说明

若出现错误信息，或没有出现需要的第几号总线，请参考 2.3 TWI 基础配置检查进行配置。

- 步骤 3

通过下面的命令查看中断，来大致确认 SPI 模块驱动是否加载成功。如没有，请按照 **<2.3 TWI 基础配置检查 >**，确认配置是否正确。

```
cat proc/interrupts
61: 2 0 0 0 wakeupgen 11 Level spi1
62: 0 0 0 0 wakeupgen 50 Level spi3
```

- 步骤 4

调试时只需要打开 DEBUG_INFO 项的打印，可以在控制台输入以下命令。

```
echo 8 > /sys/module/spi_sunxi/parameters/debug
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

1. 打印函数名为 dprintk，dprintk 通过判断模块参数 debug 的位掩码，来决定是否打印当前调试信息。参数路径为/sys/module/spi_sunxi/parameters/debug。定义的打印等级如下：

```
enum {
 DEBUG_INIT = 1U << 0, //初始化调试打印
 DEBUG_SUSPEND = 1U << 1, //休眠唤醒函数调试打印
 DEBUG_DATA = 1U << 2, //传输数据调试打印，需要配合 INFO1~4 使用
 DEBUG_INFO = 1U << 3, //普通调试打印
 DEBUG_INFO1 = 1U << 4, //cpu rx 函数调试打印
DEBUG_INFO2 = 1U << 5, //cpu tx 函数调试打印
 DEBUG_INFO3 = 1U << 6, //dma rx 函数调试打印
 DEBUG_INFO4 = 1U << 7, //dma tx 函数调试打印
};
```

2. 总线驱动的打印信息都带有前缀 [spi]，如需要查找 spi0 的打印信息，可以搜索 spi0。

3.2 基础调试手段

3.2.1 SPI 控制器基地址

全志各 SoC 型号对应的 SPI 控制器基地址寄存器如下表：

表 3-1: 全志 SoC 的 SPI 控制器基地址

SPI	SPI0	SPI1	SPI2	SPI3	SPI4
T3	0x01c05000	0x01c06000	0x01c17000	0x01c1f000	\
V5	0x05010000	0x05011000	0x05012000	0x05013000	\
A50	0x05010000	0x05011000	\	\	\
V5V200	0x05010000	0x05011000	0x05012000	0x05013000	\
T7	0x05010000	0x05011000	\	\	\
R328	0x05010000	0x05011000	\	\	\
A64	0x01c68000	0x01c69000	\	\	\
A63	0x05010000	0x05011000	\	\	\
H6	0x05010000	0x05011000	\	\	\
R100	0x05010000	0x05011000	\	\	\
H3 pro	0x05010000	0x05011000	\	\	\
A100	0x05010000	0x05011000	\	\	\
A523	0x04025000	0x04026000	0x04027000	\	\

版权所有 © 珠海全志科技股份有限公司。保留一切权利

V821	0x44025000	0x44026000	0x44027000	\	\
A733	0x02540000	0x02541000	0x02542000	0x02543000	0x02544000
T736	0x02540000	0x02541000	0x02542000	0x02543000	0x02544000
MR153	0x04025000	0x04026000	0x04027000	0x04028000	\
T153	0x04025000	0x04026000	0x04027000	0x04028000	\
MR536	0x04025000	0x04026000	0x04027000	0x04028000	0x04029000
T536	0x04025000	0x04026000	0x04027000	0x04028000	0x04029000

表 3-2: 全志 SoC CPUS 域的 SPI 控制器基地址。

SPI	R-SPI0
A733	0x07092000
T736	0x07092000
MR536	0x07092000
T536	0x07092000

3.2.2 SPI 寄存器调试

设备通过 UART 串口调试控制台或 adb shell 调试终端，通过以下步骤进行操作。

- 步骤 1

检查 PIN 的功能配置寄存器是否正确，确认相应 PIN 的功能配置为 SPI。需要查看芯片手册中关于 port 或 gpio 的文档，找到对应寄存器。如 T3 的 spi1 对应的管脚为 PA0、PA1、PA2、PA3、PA4。

```
echo [reg,reg] > /sys/class/sunxi_dump/dump;cat dump
```

1. 通过芯片的芯片手册查到基地址：

Module Name	Base Address
PIO	0x01C20800

图 3-1: SPI 基地址

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. PA 的地址：

1.4.1. PA Configure Register 0

Offset: 0x00	Register Name: PA_CFG0 Default Value: 0x7777_7777

图 3-2: PA 偏移地址

3. 读出 0x01c20800 的值：

```
echo 0x01c20800 > sys/class/sunxi_dump/dump
cat sys/class/sunxi_dump/dump
0x33333333
```

33333 和文档要求的值相同：

文档范例 A

18:16	R/W	0x7	PA4_SELECT 000: Input 010: ETXD3 100: Reserved 110: Reserved	001: Output 011: SPI1_CS1 101: GTXD3 111: IO Disable
15	/	/	/	

Ver 1.1 Allwinner Technology Co. Ltd. All Right Reserved Page 13

文档范例 B

GPIO Specification

14:12	R/W	0x7	PA3_SELECT 000: Input 010: ERXD0 100: UART2_RX 110: Reserved	001: Output 011: SPI1_MISO 101: GRXD0 111: IO Disable
11	/	/	/	
10:8	R/W	0x7	PA2_SELECT 000: Input 010: ERXD1 100: UART2_TX 110: Reserved	001: Output 011: SPI1_MOSI 101: GRXD1 111: IO Disable
7	/	/	/	
6:4	R/W	0x7	PA1_SELECT 000: Input 010: ERXD2 100: UART2_CTS 110: Reserved	001: Output 011: SPI1_CLK 101: GRXD2 111: IO Disable
3	/	/	Reserved	
2:0	R/W	0x7	PA0_SELECT 000: Input 010: ERXD3 100: UART2_RTS 110: Reserved	001: Output 011: SPI1_CS0 101: GRXD3 111: IO Disable

图 3-3: GPIO 文档要求

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 步骤 2

直接读 SPI1 的所有寄存器进行分析

```
echo 0x01c06000,0x01c07000 > sys/class/sunxi_dump/dump
cat sys/class/sunxi_dump/dump
```

说明

数据过多，此处不再列举。

3.3 基础配置检查

- 步骤 1

SPI device tree 配置如下，需要检查和注意的是**基地址、中断号和频率**。请查看芯片手册确认配置**{中断号 =[芯片手册的中断号]-32}**。

```
spi0: spi@05010000 {
    #address-cells = <1>;
    #size-cells = <0>;
    compatible = "allwinner,sun50i-spi";
      device_type = "spi0";
    reg = <0x0 0x05010000 0x0 0x1000>;
    interrupts = <GIC_SPI 54 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&clk_pll_periph0>, <&clk_spi0>;
    clock-frequency = <200000000>;
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&spi0_pins_a &spi0_pins_b>;
    pinctrl-1 = <&spi0_pins_c>;
    spi0_cs_number = <2>;
    spi0_cs_bitmap = <3>;
    status = "disabled";
};
```

- 步骤 2

1. SPI 总线配置

```
[spi0]
spi0_used      = 1
spi0_cs_number  = 2
spi0_cs_bitmap  = 3
spi0_cs0       = port:PC01<4><1><default><default>
spi0_cs1       = port:PC10<4><1><default><default>
spi0_sclk      = port:PC00<4><default><default><default>
spi0_mosi      = port:PC02<4><default><default><default>
spi0_miso      = port:PC03<4><default><default><default>
spi0_hold      = port:PC14<4><default><default><default>
spi0_wp        = port:PC13<4><default><default><default>
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[spi0_suspend]
spi0_cs0      = port:PC01<7><default><default><default>
spi0_cs1      = port:PC10<7><default><default><default>
spi0_sclk     = port:PC00<7><default><default><default>
spi0_mosi     = port:PC02<7><default><default><default>
spi0_miso     = port:PC03<7><default><default><default>
spi0_hold     = port:PC14<7><default><default><default>
spi0_wp       = port:PC13<7><default><default><default>
```

说明

spi0_suspend 是休眠时 PIN 脚的状态。如果此项没配，而 DTS 的 PIN 配置和 SYSCONFIG 的配置不是同一套 PIN，则休眠的时候会报 PIN 冲突，所以使用时需要检查此项是否配好。

2. 检查 MTD 配置

```
[spi0/spi_board0]
compatible      = "m25p80"
spi-max-frequency = 100000000
reg             = 0
spi-rx-bus-width  = 1
spi-tx-bus-width  = 1
```

- 步骤 3

检查 menuconfig 的配置。

在命令行中进入内核 linux 目录，执行 make ARCH=arm menuconfig（64 位平台执行 make ARCH=arm64 menuconfig）进入配置主界面，并按以下步骤操作：

```
Device Drivers --->
 |----<*>Memory Technology Device (MTD) support --->
 |   |----<*>SUNXI partitioning support
 |   |----<*>Caching block device access to MTD devices
 |   |----Self-contained MTD device drivers --->
 |   |   |----<*>Support most SPI Flash chips(AT26DF, M25P, W25X, ...)
 |   |----<*>SPI-NOR device support ----->(注：linux3.10没有此项选择)
 |----<*>SPI support --->
 |   | ----<*>SUNXI SPI Controller
```

3.4 SPI 常见问题排查

3.4.1 SPI 初始化失败

3.4.1.1 问题现象

sys_config 中设置使能不生效

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4.1.2 原因分析

sysconfig 的 spin_used 设为了 1，但是发现内核并没有编进 SPI 模块。这种情况有可能是采用 EMMC 方案的板子在 uboot 阶段 diasble 掉了 SPI。

3.4.1.3 排查步骤

- 步骤 1

设备上电时，按住电脑键盘的 Enter 键，进 uboot 阶段。

- 步骤 2

通过 fdt list spi* 命令查看 dts，是否使能 SPI 成功（status = “okay”），如果还是 disable，可以通过我司 FAE 跟负责 uboot 的同事沟通。

```
sunxi#fdt list spi1
spi@01c06000 {
  #address-cells = <0x00000001>;
  #size-cells = <0x00000000>;
  compatible = "allwinner,sun8i-spi";
  device_type = "spi1";
  reg = <0x00000000 0x01c06000 0x00000000 0x00001000>;
  interrupts = <0x00000000 0x0000000b 0x00000004>;
  clocks = <0x00000004 0x00000056>;
  clock-frequency = <0x05f5e100>;
  pinctrl-names = "default", "sleep";
  pinctrl-1 = <0x00000059>;
  spi1_cs_number = <0x00000002>;
  spi1_cs_bitmap = <0x00000003>;
  status = "okay";
  pinctrl-0 = <0x000000f9 0x000000fa>;
  spi_board0 {
  };
};
```

3.4.2 SPI Norflash 兼容性问题

3.4.2.1 问题现象

在/dev 中没有找到 mtdblockn，启动时 log 报有 “unrecognized JEDEC id bytes:”

```
[ 2.127716] m25p80 spi1.0: unrecognized JEDEC id bytes: 00, 00, 00
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4.2.2 原因分析

nor flash 的规格参数没有加到 spi_nor_ids 结构体中。

3.4.2.3 排查步骤

- 步骤 1

查找 flash 数据手册，找到 ID Definitions 表，得到 Manufacturer ID、Memory type、Memory density，并在数据手册 features 中确认容量，此款 flash 是否支持 4K 擦除、双线读和四线读功能。

Table 6. ID Definitions

Command Type		MX25L12835F		
RDID	9Fh	Manufacturer ID	Memory type	Memory density
		C2	20	18

图 3-4: ID Definitions

步骤 2：

在内核 driver/mtd/spi-nor/spi-nor.c 的 spi_nor_ids 结构体中添加上新的 flash INFO。

```
{ "mx25l12835f", INFO(0xc22018, 0, 64 * 1024, 256, SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ) },
```

图 3-5: 添加新的 flash

3.4.3 SPI 数据传输异常

3.4.3.1 问题现象

写入与读出数据不一致。

3.4.3.2 排查步骤

- 步骤 1

进行兼容性排查。以 nor flash 为例，有些物料兼容性不好，会造成读写出错。这时可以先确认下此款物料是否在支持列表内。若不在，试着更换物料再做测试。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 步骤 2

驱动调试。此类问题范围比较大，但是可以从基础调试手段着手跟踪调试。一般思路是打开数据打印，看写入的值是否传到 SPI 总线驱动处理，然后同样的看 SPI 总线驱动刚读出来的数据与前面写的打印数据是否一致，来判断是哪个环节造成读写出错，这个办法可以拓展到其他层次，以确认是文件系统层、MTD 层、SPI 总线驱动层的读或写问题。

下图是使用 nor 测试用例裸测 mtdblock 分区的实验示例。未开打印的如下：

```
/tmp # ./nor_write 64 0x55 /dev/mtdblock5
/tmp # ./nor_read 64 /dev/mtdblock5
-----------------------------------------------------------------
0x00000000: 55 55 55 55 55 55 55 55    55 55 55 55 55 55 55 55
0x00000010: 55 55 55 55 55 55 55 55    55 55 55 55 55 55 55 55
0x00000020: 55 55 55 55 55 55 55 55    55 55 55 55 55 55 55 55
0x00000030: 55 55 55 55 55 55 55 55    55 55 55 55 55 55 55 55
-----------------------------------------------------------------
/tmp #
```

图 3-6: nor 测试用例裸测 mtdblock 分区-未开打印

开了打印的实验如下：

```
/tmp # echo 36 > /sys/module/spi_sunxi/parameters/debug
/tmp # ./nor_write 64 0x55 /dev/mtdblock5
[  654.835153] 000: 6b f2 00 00 00
[  654.839013] 000: 6b f2 02 00 00
[  654.842848] 000: 6b f2 04 00 00
[  654.846596] 000: 6b f2 06 00 00
[  654.850337] 000: 6b f2 08 00 00
[  654.854129] 000: 6b f2 0a 00 00
[  654.857872] 000: 6b f2 0c 00 00
[  654.861661] 000: 6b f2 0e 00 00
[  654.865674] 000: 6b f2 00 00 00
[  654.869692] 000: 06
[  654.872255] 000: 20 f2 00 00
[  654.875646] 000: 05
[  654.878163] 000: 05
[  654.880676] 000: 05
[  654.883251] 000: 05
[  654.885757] 000: 05
[  654.888284] 000: 05
[  654.890802] 000: 05
[  654.893341] 000: 05
```

图 3-7: nor 测试用例裸测 mtdblock 分区-开启打印

上面的实验只是本调试方法手段其中的一种。以上图为例，

echo 36 > /sys/module/spi_sunxi/parameters/debug

实际开的是 cpu tx 的数据打印，SPI 驱动设定传输字节数小于 64cpu 传输，大于或等于 64 字节用 DMA 传输。nor 写地址，写寄存器的字节数都小于 64 字节。上图中 “6b f2 00 00 00” 表示为四

版权所有 © 珠海全志科技股份有限公司。保留一切权利

线写操作码 6b，nor 地址为 f2 00 00 00。以此类推，需要那一部分的数据，则打开那一部分的打印，详细参照 **<3.1 节打印等级定义 >**。

3.4.4 SPI 数据传输首字节丢失

3.4.4.1 问题现象

spi-flash 写入与读出数据不一致。发现首字节丢失两个 bit 并被错误写到前一个地址。

3.4.4.2 排查步骤

- 步骤 1

通过多次实验发现只有对 flash 奇数地址写入数据会发生这个问题，使用示波器查看波形，发现数据最后一个 bit 高电平只维持了半个 clock 的时间。对于某些对时序要求比较高的 flash 会有影响，因此造成传输错位，经设计沟通全平台 spi mode0 都存在该问题。

- 步骤 2

通过使用其他 spi mode 可以规避此问题，或者将 SPI Transfer Control Register 寄存器（偏移值为 0x08）的 bit 14（SDDM）置为 1，可以解决此问题。

3.4.5 SPI 传输超时无响应

3.4.5.1 问题现象

SPI 传输大量数据时，出现 timeout

3.4.5.2 排查步骤

- 步骤 1

SPI 出现 timeout，一般都是 SPI 中断无响应造成的，主要是分析出无响应的原因。可以查看 0x10（SPI Interrupt Control）寄存器里 SPI 中断位（bit12）是否被正确使能，亦可同时查看 0x14（SPI Interrupt Status）寄存器确定中断状态的异常情况。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 步骤 2

SPI 传输可选择 CPU 传输或者 DMA 传输，先确定仅使用 CPU 传输或仅使用 DMA 传输有无问题。

- 步骤 3

如果出现 timeout 发生在 DMA 传输，可以多关注 0x18（SPI FIFO Control）寄存器，可能是传输途中 FIFO DMA 被意外关闭了，在一些较老版本中，SPI 完整一包传输会关闭。在 sunxi_spi_dma_cb_rx、sunxi_spi_dma_cb_tx 函数里注释掉 spi_disable_dma_irq 语句即可。

3.4.6 SPI 传输 CS 提前被拉低

3.4.6.1 问题现象

SPI 传输时，CS 脚被提前拉低 100us 左右，现象如图：

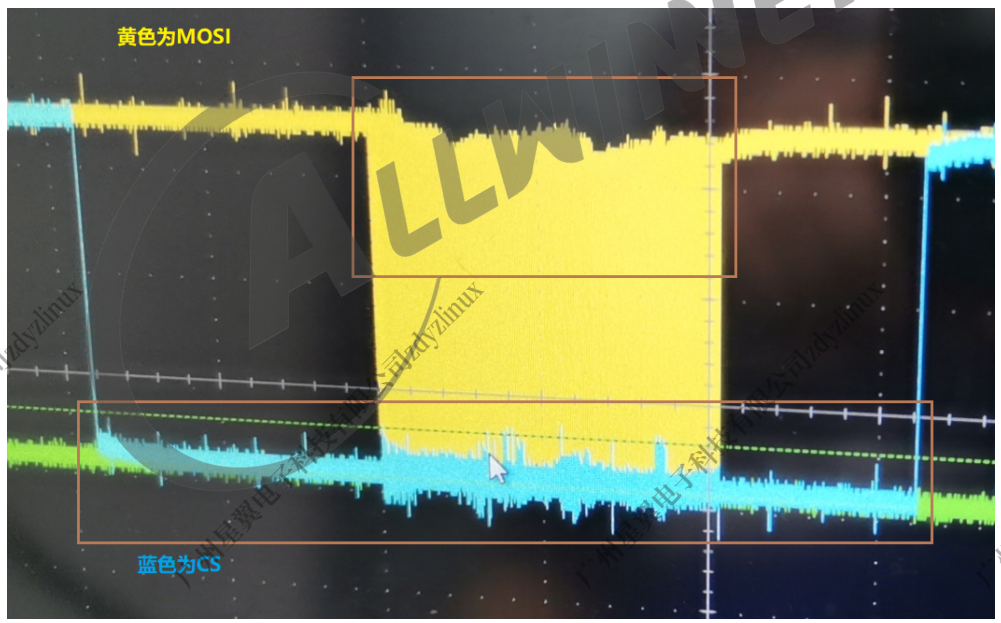

图 3-8: SPI cs 被提前拉低 100us

3.4.6.2 排查步骤

- 步骤 1

该问题可能是由于使用 SPI 内部分频造成的。经设计确认，使用 SPI 内部分频寄存器（0x24）会导致此问题，sunxi-dev 分支下 linux-4.9 已解决此问题，参考 spi_set_clk 函数内部实现即可。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

步骤 2：

如果问题没有解决，可以将 CS 脚配置由 SPI Control 控制，sunxi-dev 默认使用软件控制 CS：

spi_ss_ctrl(base_addr, 1);

修改为：

spi_ss_ctrl(base_addr, 0);

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 USB

4.1 日志信息调试

- 步骤 1

样机设备连接 UART 串口调试线或者 adb 调试线。

- 步骤 2

通过串口 console 调试终端或者 adb shell 调试终端输入以下命令，提高内核信息打印等级，串口 console 可以正常看到内核的日志信息。Adb shell 调试终端需要输入 dmesg 命令查看内核的日志信息。

```
echo 8 > /proc/sysrq-trigger
echo 8 > /proc/sys/kernel/printk
```

- 步骤 3

样机的 USB 口插入 USB 设备，如果样机设备正常识别到 USB 设备，可以看到类似以下的日志信息（以 U 盘为例）。

```
[93355.022561] sunxi-ehci 5101000.ehci0-controller: ehci_irq: highspeed device connect
[93355.321193] usb 1-1: new high-speed USB device number 5 using sunxi-ehci
[93355.480272] usb 1-1: New USB device found, idVendor=0930, idProduct=6545
[93355.488168] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[93355.496521] usb 1-1: Product: DataTraveler 2.0
[93355.501805] usb 1-1: Manufacturer: Kingston
[93355.506624] usb 1-1: SerialNumber: 1C6F654E4910B15019173AD2
[93355.518964] usb-storage 1-1:1.0: USB Mass Storage device detected
[93355.530515] scsi host0: usb-storage 1-1:1.0
[93356.569002] scsi 0:0:0:0: Direct-Access     Kingston DataTraveler 2.0 PMAP PQ: 0 ANSI: 6
[93356.591814] sd 0:0:0:0: [sda] 30240768 512-byte logical blocks: (15.5 GB/14.4 GiB)
[93356.601795] sd 0:0:0:0: [sda] Write Protect is off
[93356.607332] sd 0:0:0:0: [sda] Mode Sense: 45 00 00 00
[93356.613890] sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
[93356.737282]  sda: sda1
[93356.750383] sd 0:0:0:0: [sda] Attached SCSI removable disk
```

- 步骤 4

版权所有 © 珠海全志科技股份有限公司。保留一切权利

如果样机 USB 口插入 USB Storage 设备，可以通过 mount 命令检查 USB 设备是否挂载成功。

输入命令

```
mount
```

输出信息

```
/dev/block/vold/public:8,1 on /mnt/media_rw/B49F-D07E type vfat
(rw,dirsync,nosuid,nodev,noexec,noatime,uid=1023,gid=1023,fmask=0007,dmask=0007,
allow_utime=0020,codepage=437,iocharset=iso8859-1,shortname=mixed,utf8,errors=remount-ro)

/mnt/media_rw/B49F-D07E on /mnt/runtime/default/B49F-D07E type sdcardfs
(rw,nosuid,nodev,noexec,noatime,fsuid=1023,fsgid=1023,gid=1015,mask=6)

/mnt/media_rw/B49F-D07E on /storage/B49F-D07E type sdcardfs
(rw,nosuid,nodev,noexec,noatime,fsuid=1023,fsgid=1023,gid=1015,mask=6)

/mnt/media_rw/B49F-D07E on /mnt/runtime/read/B49F-D07E type sdcardfs
(rw,nosuid,nodev,noexec,noatime,fsuid=1023,fsgid=1023,gid=9997,mask=18)

/mnt/media_rw/B49F-D07E on /mnt/runtime/write/B49F-D07E type sdcardfs
(rw,nosuid,nodev,noexec,noatime,fsuid=1023,fsgid=1023,gid=9997,mask=18)
```

- 步骤 5

如果样机的 USB 口插入 USB 接口类设备，可以通过命令 lsusb 检查 Usb 设备是否注册成功。

插入设备前：

```
#lsusb
```

输出打印：

```
Bus 001 Device 001: ID 1d6b:0002
Bus 002 Device 001: ID 1d6b:0001
```

插入 usb 设备后：

```
#lsusb
```

输出打印：

```
Bus 001 Device 001: ID 1d6b:0002
Bus 001 Device 005: ID 0c45:6366 // 新增
Bus 002 Device 001: ID 1d6b:0001
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 基础调试手段

4.2.1 USB 控制器基地址

全志科技各个 SoC 型号对应的 USB 控制器基地址寄存器如下表：

表 4-1: 各个 SoC 型号对应的 USB 控制器基地址寄存器表。

	Device	Host			
	USB_DRD_Device	USB_DRD_Host	USB_HCI1	USB_HCI2	USB_HCI3
H3	0x01c19000	0x01c1a000	0x01c1b000	0x01c1c000	0x01c1d000
V3	0x01c19000	0x01c1a000	\	\	\
A64	0x01c19000	0x01c1a000	0x01c1b000	\	\
B100	0x01c19000	0x01c1a000	\	\	\
V40	0x01c13000	0x01c14000	0x01c19000	0x01c1c000	\
H5	0x01c19000	0x01c1a000	0x01c1b000	0x01c1c000	0x01c1d000
H6	0x05100000	0x05101000	\	\	\
A63	0x05100000	0x05101000	0x05200000	\	\
V5	0x05100000	0x05101000	0x05200000	\	\
T7	0x05100000	0x05101000	0x05200000	0x05310000	0x05311000
A50	0x05100000	0x05101000	0x05200000	\	\
A523	0x04100000	0x04101000	0x04200000	\	\
A733	0x04100000	0x04101000	0x04200000	\	\
T736	0x04100000	0x04101000	0x04200000	\	\
MR153	0x04200000	0x04201000	0x04300000	\	\
T153	0x04200000	0x04201000	0x04300000	\	\
MR536	0x04100000	0x04101000	0x04200000	\	\
T536	0x04100000	0x04101000	0x04200000	\	\

说明

具体或未列出的型号请参考对应芯片手册。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.2 高速眼图测试命令

更多详细信息可参考《Linux_USB_ 开发指南》中的 “眼图测试” 章节。以下简略介绍眼图测试方法：

4.2.2.1 USB Device 眼图测试

- 步骤 1

寻找 **otg_ed_test** 节点。

```
find ./ -name otg_ed_test
```

- 步骤 2

根据寻找到的路径，向节点 otg_ed_test 写入 test_pack。

```
echo test_pack > /sys/devices/.../otg_ed_test
```

4.2.2.2 USB Host 眼图测试

- 步骤 1

寻找 **ed_test** 节点。

```
find ./ -name ed_test
```

- 步骤 2

根据寻找到的路径，选择需要测试的 usb 口，向其对应的节点 ed_test 写入 test_pack。

例如：测试 usb1 口的眼图，如下

```
echo test_pack > /sys/devices/.../xxxx.ehci1-controller/ed_test
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.3 USB Host 驱动能力调节

更多详细信息可参考《Linux_USB_ 开发指南》中的 “驱动能力调节” 章节。以下简略介绍驱动能力调节方法：

- 步骤 1

寻找 phy_range 节点；

```
find ./ -name phy_range
```

- 步骤 2

根据寻找到的路径，读取异常 usb 口对应的 phy_range 值；

```
cat /sys/devices/xxx/.../phy_range
```

- 步骤 3

调节 phy_range 值，低两位是幅度，高三位是斜率，具体修改值需请教硬件相关同事；

```
echo XXXX > /sys/devices/xxx/.../phy_range
```

- 步骤 4

确认 phy_range 值修改成功。

```
cat /sys/devices/xxx/.../phy_range
```

4.2.4 USB 寄存器打印

寄存器打印的命令形式如下：

```
echo 起始地址，结束地址 > /sys/class/sunxi_dump/dump
cat /sys/class/sunxi_dump/dump
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.4.1 USB Device 寄存器打印

- 步骤 1

根据 USB 控制器基地址或相关芯片手册确认基地址 Base Address；

- 步骤 2

打印出偏移为 0x40~0x98，0x400~0x424 的寄存器值。

4.2.4.2 USB Host 寄存器打印

- 步骤 1

根据 USB 控制器基地址或相关芯片手册确认需要打印的 USB 口的基地址 Base Address；

- 步骤 2

打印偏移为 0x00~0x54，0x400~0x454，0x800~0x830 的寄存器值。

4.3 基础配置检查

具体配置释义可参考《Linux_USB_开发指南》中的 “模块配置介绍” 章节。

4.4 USB 常见问题排查

4.4.1 USB 硬件完备排查

4.4.1.1 USB Device 硬件完备排查

USB Device 硬件完备排查主要分为 “芯片级排查” 和 “板级排查”。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.4.1.1.1 芯片级排查

- 步骤 1

确认供电是否正常。

1. VCC-USB 为 3.3V±10%。
2. 对于 40nm 平台，VDD-USB 为 1.1V±10%。
3. 对于 28nm 平台，VDD-USB 为 0.9V±10%。

- 步骤 2

确认 usb device 控制信号是否正常，具体请查看相关芯片手册。

1. 复位和门控信号由 CCU 控制，确认配置是否正确。
2. 中断信号由 GIC 控制，确认中断号和使能控制是否正确。
3. 若使用系统 DMA，确认 DRQ 号和使能控制是否正确。

- 步骤 3

确认时钟是否正常。

1. OTG 所在总线的时钟，必须保证大于 30MHz，确认是否被修改过。
2. PHY_SCLK 源于 HOSC24M，确认 HOSC24M 是否正常。

- 步骤 4

确认其他杂项是否正常。

1. 确认 AVCC 供电是否正常。
2. 对于 40nm 平台，确认 VRA1 电压是否等于 AVCC/2。
3. 对于 28nm 平台，确认 REXT 电压是否为 0.75V。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.4.1.1.2 板级排查

- 步骤 1

确认 USB 信号的连通是否正常。DP、DM、ID、VBUS 和 GND 连接是否正确，焊接是否可靠。

- 步骤 2

确认 VBUS 和 ID 的检测电路，是否与标案一致，是否有不适当的修改。

- 步骤 3

确认 USB 设备初始化后 DP、DM 的电平。DP 应该有 1.5K 电阻上拉到 VCC-USB，DM 为低电平。

- 步骤 4

确认 USB 信号的走线是否正常。重点确认 DP、DM 的走线有无分叉、是否差分走线等。

- 步骤 5

确认 ESD 器件是否正常。重点确认 DP、DM 上的 ESD 器件，是否满足应用要求。

4.4.1.2 USB Host 硬件完备排查

USB Host 硬件完备排查主要分为 “芯片级排查” 和 “板级排查”。

4.4.1.2.1 芯片级排查

- 步骤 1

确认 usb 供电是否正常。

1. VCC-USB 为 3.3V±10%。
2. 对于 40nm 平台，VDD-USB 为 1.1V±10%。
3. 对于 28nm 平台，VDD-USB 为 0.9V±10%。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 步骤 2

确认 usb host 控制信号是否正常，具体请查看相关芯片手册。

1. 复位和门控信号由 CCU 控制，确认配置是否正确。
2. 中断信号由 GIC 控制，确认中断号和使能控制是否正确。

- 步骤 3

确认 usb host 时钟是否正常。具体请查看相关芯片手册。

1. EHCI 和 OHCI 所在总线的时钟，确认是否被修改过。
2. OHCI_SCLK_48M/12M 源于外设 PLL_PERI0 采样 HOSC24M 得到的 48M 时钟，确认 PLL_PERI0 是否被修改过、HOSC24M 是否正常。
3. PHY_SCLK 源于 HOSC24M，确认 HOSC24M 是否正常。

- 步骤 4

确认其他杂项是否正常。

1. 确认 AVCC 供电是否正常。
2. 对于 40nm 平台，确认 VRA1 电压是否等于 AVCC/2。
3. 对于 28nm 平台，确认 REXT 电压是否为 0.75V。

4.4.1.2.2 板级排查

- 步骤 1

确认 USB 信号的连通是否正常。DP、DM、ID、VBUS 和 GND 连接是否正确，焊接是否可靠。

- 步骤 2

确认 VBUS 供电是否正常。VBUS 为 5V±10%。

- 步骤 3

版权所有 © 珠海全志科技股份有限公司。保留一切权利

确认 USB 设备初始化后 DP、DM 的电平。DP、DM 应该都有 15K 电阻下拉到 GND。

- 步骤 4

确认 USB 信号的走线是否正常。重点确认 DP、DM 的走线有无分叉、是否差分走线等。

- 步骤 5

确认 ESD 器件是否正常。重点确认 DP、DM 上的 ESD 器件，是否满足应用要求。

4.4.2 USB 基本功能异常排查

更多的功能异常案例排查示例，请参考《Linux_USB_ 开发指南》。

4.4.2.1 USB Device 基本功能异常

- 步骤 1

多换几个 PC 主机做相同的测试，排除个别 PC 的问题。

- 步骤 2

多更换几根 USB 线缆做相同的测试，排除个别 USB 线缆的问题。

- 步骤 3

确认 device 驱动是否加载成功，可通过如下方式：

方式 1：查看 log，有如下关键信息：

```
[ 104.732695] insmod_device_driver
[ 104.732695] device_chose finished!
```

方式 2：若 usb0 口配置为 otg，可通过如下命令确认当前 usb0 的 role。

```
cat `busybox find /sys/devices/ -name "otg_role"`
```

- 步骤 4

重新加载 device 驱动，确认此时功能是否恢复正常。

方式 1：重新插拔 usb 线；

方式 2：cat usb_device 节点。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
cat `busybox find /sys/devices/ -name "usb_device"`
```

- 步骤 5

对比客户 SDK 和 sunxi-dev 分支 sunxi_usb 目录下的代码，确认是否由于代码差异导致该问题。

- 步骤 6

同样条件下，参考 USB Device 寄存器打印，分别打印出功能异常板子和功能正常板子的相关寄存器，并进行对比。对寄存器不同之处查看对应芯片手册，确认是否有影响。

- 步骤 7

出现异常时，确认 USB 高速眼图是否正常，请参考 Device 模式眼图测试。

- 步骤 8

参考下图将设备配置成全速模式，确认功能是否正常。

```
static void sunxi_udc_enable(struct sunxi_udc *dev)
{
        DMSG_DBG_UDC("sunxi_udc_enable called\n");

        dev->gadget.speed = USB_SPEED_UNKNOWN;

        DMSG_INFO_UDC("CONFIG_USB_GADGET_DUALSPEED: USBC_TS_MODE_HS\n");

        USBC_Dev_ConfigTransferMode(
                                g_sunxi_udc_io.usb_bsp_hdle,
                                USBC_TS_TYPE_BULK,
                                USBC_TS_MODE_HS);
```

USBC_TS_MODE_FS

图 4-1: 将设备配置成全速模式

- 步骤 9

出现异常时，对硬件进行完备排查，请参考 USB Device 硬件完备排查。

- 步骤 10

若上述步骤均未发现，通过我司 FAE 将结果反馈给模块负责人。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.4.2.2 USB Host 基本功能异常

- 步骤 1

多找几个 USB 设备试试，排除个别 USB 设备本身的问题。

- 步骤 2

多更换几根 USB 线缆试试，排除个别 USB 线缆的问题。

- 步骤 3

多找几个 PC 主机做相同的实验，作为参考对比。若在 PC 有相同现象，则认为正常。

- 步骤 4

样机设备 USB 口外接独立供电的 usb-hub 设备，再将 usb 设备连接到 usb-hub 上，确认主机功能是否正常。

- 步骤 5

确认主机驱动是否加载成功。

1. 若为 usb0 口，则可通过如下方式确认

```
cat `busybox find /sys/devices/ -name "otg_role"`
```

2. 若为其他 usb 口（非 usb0 口），需查看启动完整 log。关键信息为：

```
[3.112038] [sunxi-ehci1]: probe, pdev->name: 5200000.ehci1-controller, sunxi_ehci: 0xc0b35b00, 0x:f145c000, irq_no:cb
[3.124210] sunxi-ehci 5200000.ehci1-controller: SW USB2.0 'Enhanced' Host Controller (EHCI) Driver
[3.134616] sunxi-ehci 5200000.ehci1-controller: new USB bus registered, assigned bus number 1
[3.145618] sunxi-ehci 5200000.ehci1-controller: irq 203, io mem 0xc0b35b00
[3.180572] sunxi-ehci 5200000.ehci1-controller: USB 0.0 started, EHCI 1.00
[3.196563] hub 1-0:1.0: USB hub found
[3.201036] hub 1-0:1.0: 1 port detected
```

- 步骤 6

重新加载 host 驱动，确认此时功能是否正常；

1. 若为 usb0 口，则可通过如下方式：

方式 i：重新拔插 otg 线；

方式 ii：cat usb_host 节点。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
cat `busybox find /sys/devices/ -name "usb_host"`
```

2. 若是其他 usb 口（非 usb0 口），则可通过如下方式：

i. 确认异常 usb 口的 ehci_enable 路径；

```
find ./ -name ehci_enable
```

ii. 依次执行向异常 usb 口对应的 ehci_enable 节点写 0 再写 1。

```
echo 0 >/xxx/.../ehci_enable
echo 1 >/xxx/.../ehci_enable
```

- 步骤 7

若硬件有多个 usb 口，尝试同样条件下测试其他 usb 口的主机功能是否正常。

- 步骤 8

对比客户 SDK 和 sunxi-dev 分支 host 目录下的代码，确认是否有差异。若有差异，确认是否是由差异引起的功能异常。

- 步骤 9

同样条件下，参考 USB Host 寄存器打印，分别打印出功能异常板子和功能正常板子的相关寄存器，并进行对比。对寄存器不同之处查看对应芯片手册，确认是否有影响。

- 步骤 10

出现异常时，测试 USB 高速眼图是否正常。请参考 USB Host 眼图测试。

- 步骤 11

若眼图测试未通过，可尝试调节眼图参数，请参考 USB Host 硬件完备排查。

- 步骤 12

版权所有 © 珠海全志科技股份有限公司。保留一切权利

尝试用全速设备和低速设备做相同的测试，确认主机功能是否正常。

- 步骤 13

出现异常时，对硬件进行完备排查。请参考 USB Host 硬件完备排查。

- 步骤 14

若上述步骤均未发现，将结果通过我司 FAE 反馈给模块负责人。

4.4.2.3 USB 同步传输速率说明

理论上，usb2.0 的传输速率为 480Mbps，即 60MB/s。但实际上考虑到协议开销等因素的影响，usb 2.0 的传输速率大致为 40MB/s，其中**预留给同步传输的带宽是 24M**，其他的设备共享剩余的 16M 带宽，**并且会优先保证同步传输的带宽**。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 IR-RX

5.1 日志信息调试

- 步骤 1

设备通过 UART 串口线连接调试控制台，或使用 usb 连接使用 adb shell 进行调试。

- 步骤 2

开启设备完成后，通过下面的命令查看是否有如下图 sunxi-ir 名称的 input 设备。若没有，请参考 **5.3IR-RX 基础配置检查**进行配置。

```
cat /proc/bus/input/devices
```

```
/ # cat proc/bus/input/devices
I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="sunxi-keyboard"
P: Phys=sunxikbd/input0
S: Sysfs=/devices/virtual/input/input0
U: Uniq=
H: Handlers=kbd event0
B: PROP=0
B: EV=3
B: KEY=800 c0040 0 0 10000000

I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="sunxi-ir"
P: Phys=sunxi-ir/input0
S: Sysfs=/devices/platform/soc/ir0/rc/rc0/input1
U: Uniq=
H: Handlers=kbd event1
B: PROP=0
B: EV=100013
B: KEY=2
B: MSC=10
```

图 5-1: 查看红外接收设备

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2 基础调试手段

- 步骤 1

打开设备

```
/ # cd dev/input/
/dev/input # ls
event0  event1  event2  event3  mice    mouse0
```

图 5-2: 打开设备

- 步骤 2

根据 **5.1** 中查看的 Handlers，使用 hexdump 命令 (如果是 android 平台，使用 getevent 命令)，接着按下遥控器按键，会报出类似以下数据。

```
/dev/input # hexdump event1
0000000 036d 0000 fdae 0001 0004 0004 0410 00fb
0000010 036d 0000 fdae 0001 0000 0000 0000 0000
0000020 036d 0000 1717 000b 0004 0004 0411 00fb
0000030 036d 0000 1717 000b 0000 0000 0000 0000
```

图 5-3: hexdump 日志

若无数据，请查看硬件是否有问题？是否能进入红外接收中断处理？

- 步骤 3

先查看红外接收的中断次数，再按遥控器，接着再查看，观察次数是否有增加？

```
/ # cat proc/interrupts
           CPU0       CPU1       CPU2       CPU3
 17:          0          0          0          0  wakeupgen  22 Level     sun4i_timer0
 18:          0          0          0          0     GIC-0  29 Level     arch_timer
 19:      12413        769        517       1111     GIC-0  30 Level     arch_timer
 53:          0          0          0          0  wakeupgen  27 Level     1c02000.dma-controller
 54:          0          0          0          0  wakeupgen  24 Level     rtc
 55:        807          0          0          0  wakeupgen   5 Level     RemoteIR_RX
 56:          0          0          0          0  wakeupgen  53 Level     cedar_dev
```

图 5-4: 查看红外接收的中断次数 1

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
/ # cat proc/interrupts
           CPU0       CPU1       CPU2       CPU3
 17:          0          0          0          0  wakeupgen  22 Level     sun4i_timer0
 18:          0          0          0          0     GIC-0  29 Level     arch_timer
 19:      12452        784        527       1113     GIC-0  30 Level     arch_timer
 53:          0          0          0          0  wakeupgen  27 Level     1c02000.dma-controller
 54:          0          0          0          0  wakeupgen  24 Level     rtc
 55:        867          0          0          0  wakeupgen   5 Level     RemoteIR_RX
 56:          0          0          0          0  wakeupgen  53 Level     cedar_dev
```

图 5-5: 查看红外接收的中断次数 2

5.3 基础配置检查

全志各 SoC 型号对应的 IR-RX 控制器基地址寄存器如下表：

表 5-1: 全志 SoC 的 IR-RX 控制器基地址。

IR/S_CIR	0	1	2	3	R
T3	0x01c21c00	\	\	\	\
V5	0x07040000	\	\	\	\
A50	0x07040000	\	\	\	\
V5V200	0x07040000	\	\	\	\
T7	0x07040000	\	\	\	\
A64	0x01f02000	\	\	\	\
H6	0x07040000	\	\	\	\
R100	0x07040000	\	\	\	\
H3 pro	0x07040000	\	\	\	\
A523	0x02005000	\	\	\	0x07040000
A733	0x02526000	\	\	\	0x07040000
T736	0x02526000	\	\	\	0x07040000
MR153	0x02081000	0x02082000	0x02083000	0x02084000	\
T153	0x02081000	0x02082000	0x02083000	0x02084000	\
MR536	0x02081000	0x02082000	0x02083000	0x02084000	0x07040000
T536	0x02081000	0x02082000	0x02083000	0x02084000	0x07040000

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 步骤 1

s_cir0 device tree 配置如下，需要检查和注意的是基地址、中断号、引脚。请查看芯片手册确认配置 **{中断号 =[芯片手册的中断号]-32}**。

```
//sun***-dtsi
s_cir0: s_cir@07040000 {
    compatible = "allwinner,s_cir";
    reg = <0x0 0x07040000 0x0 0x400>;
    interrupts = <GIC_SPI 13 IRQ_TYPE_LEVEL_HIGH>;
    pinctrl-names = "default";
    pinctrl-0 = <s_cir0_pins_a>;
    clocks = <&clk_hosc>,<&clk_cpurcir>;
    supply = "vcc-pl";
    supply_vol = "3300000";
    status = "okay";
};

//sun***-pinctrl.dtsi
s_cir0_pins_a: s_cir0@0 {
    allwinner,pins = "PL9";
    allwinner,function = "s_cir0";
    allwinner,muxsel = <2>;
    allwinner,drive = <2>;
    allwinner,pull = <1>;
};
```

- 步骤 2

sysconfig 包含使能和控制方式配置。

```
[s_cir0]
s_cir0_used        = 1
ir_protocol_used   = 0
ir_power_key_code0 = 0x57
ir_addr_code0      = 0x9f00
```

其中：

1. s_cir0_used 为 1 表示使能 S_CIR0 控制器，0 表示不使能；
2. ir_protocol_used 为 0 表示 NEC 协议，1 表示 RC5 协议；
3. ir_power_key_code0 是解码后上报的键值；
4. ir_addr_code0 是红外发送的地址码；

- 步骤 3

menuconfig 的配置。

在命令行中进入内核 linux 目录，执行 make ARCH=arm menuconfig（64 位平台执行 make ARCH=arm64 menuconfig）进入配置主界面，并按以下步骤操作：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Device Drivers --->
  |----<*>Multimedia support --->
  |   | ----[*]Remote Controller support
  |   | ----[*]Remote Controller decoders
  |   |   |----<*>Enable IR raw devoder for the NEC protocol (NEW)
  |   |   |----<*>Enable IR raw devoder for the RC-5 protocol
  |   | ----[*]Remote Controller devices
  |   |   | ----<*>Sunxi IR remote control
```

5.4 IR-RX 常见问题排查

5.4.1 android 平台操作红外失效排查手段

5.4.1.1 问题现象

无法通过红外遥控操作机器。

5.4.1.2 排查步骤

- 步骤 1

按下遥控，查看是否有红外遥控中断触发。通过/proc/interrupts 判断，如果中断次数在遥控操作过程中没有增加，优先怀疑是板子硬件问题，例如红外头损坏，或者是遥控损坏。

```
# cat /proc/interrupts
CPU0
......
208: 34 GIC-0 141 Level RemoteIR_RX
```

- 步骤 2

通过上面的判断，IR 中断能正常触发，则通过 getevent 查看是否有按键正常上报：

```
# getevent
add device 1: /dev/input/event2
name: "sunxi-ir-uinput"
add device 2: /dev/input/event1
name: "sunxi-ir"
add device 3: /dev/input/event0
name: "sunxi-keyboard"
/dev/input/event1: 0004 0004 01fe0150
/dev/input/event1: 0000 0000 00000000
/dev/input/event2: 0001 0016 00000001 //0001 表示EV\_KEY事件
/dev/input/event2: 0000 0000 00000000
/dev/input/event2: 0001 0016 00000000
/dev/input/event1: 0004 0004 00fe0150
/dev/input/event2: 0000 0000 00000000
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
/dev/input/event1: 0000 0000 00000000
```

sunxi 平台，操作红外操作正常情况下，会有两个 input 设备上报数据，设备名为 sunxi-ir 和 sunxi-ir-uinput。

没有 sunxi-ir-uinput 设备

如果没有 sunxi-ir-uinput 设备，则需要检查 multi-ir 进程是否正常跑起？出现概率较低。

```
# ps | grep -i multi_ir
system 1873 1 4820 2056 poll_schedule_timeout 0 S multi_ir
```

有 sunxi-ir-uinput 设备，但是按下遥控，没有 sunxi-ir-uinput 设备事件上报

```
# getevent
add device 1: /dev/input/event2
name: "sunxi-ir-uinput"
add device 2: /dev/input/event1
name: "sunxi-ir"
add device 3: /dev/input/event0
name: "sunxi-keyboard"
/dev/input/event1: 0014 0000 00000000
/dev/input/event1: 0014 0001 00000000
/dev/input/event1: 0004 0004 01**ff00**15
/dev/input/event1: 0000 0000 00000000
/dev/input/event1: 0004 0004 00ff0015
/dev/input/event1: 0000 0000 00000000
/dev/input/event1: 0004 0004 01ff0015
/dev/input/event1: 0000 0000 00000000
/dev/input/event1: 0004 0004 00ff0015
```

一般由于没有 customer_ir_xxxx.kl，其中 xxxx 对应的是上面 log 中加粗字段的数字，该文件由遥控厂商配置过来。该例子将 customer_ir_ff00.kl 放置/system/usr/keylayout/目录下即可。

```
console:/system/usr/keylayout # ls
customer\_ir\_2992.kl
customer\_ir\_4cb3.kl
customer\_ir\_9f00.kl
customer\_ir\_bc00.kl
customer\_ir\_dd22.kl
customer\_ir\_fb04.kl
customer\_ir\_fc00.kl
customer\_ir\_fe01.kl
```

有 sunxi-ir-uinput 和 sunxi-ir 设备事件上报，但操作失效

该类现象一般有两种问题导致:

其一是 customer_ir_xxxx.kl 配置错误，导致映射混乱，这种情况需要配置正确的 customer_ir_xxxx.kl 文件；

其二是 sunxi-ir 没有正确匹配正常的 kl 文件，通过如下操作查看：

```
console:/system/usr/keylayout \# dumpsys input | grep -i sunxi-ir
2: sunxi-ir
Location: sunxi-ir/input0
KeyLayoutFile: /system/usr/keylayout/sunxi-ir.kl
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Device 1: sunxi-ir-uinput
Device 2: sunxi-ir
```

正常 sunxi-ir 设备匹配的是/system/usr/keylayout/sunxi-ir.kl。如果匹配错误，可能是/system/usr/keylayout/Generic.kl。这种情况需要确认/system/usr/keylayout/sunxi-ir.kl 是否存在并且有访问权限；如果该文件存在并由权限，则需要看下该文件是否自己定义了键值，在 framworks 里面没有匹配。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 LRADC（Keyboard）

6.1 日志信息调试

- 步骤 1

设备通过 UART 串口线连接调试控制台，或使用 usb 连接使用 adb shell 进行调试。

- 步骤 2

开启设备完成后，通过以下命令查看是否有如下图 sunxi-keyboard 名称的 input 设备。若没有，请参考 **6.3LRADC 基础配置检查**进行配置。

```
cat /proc/bus/input/devices
```

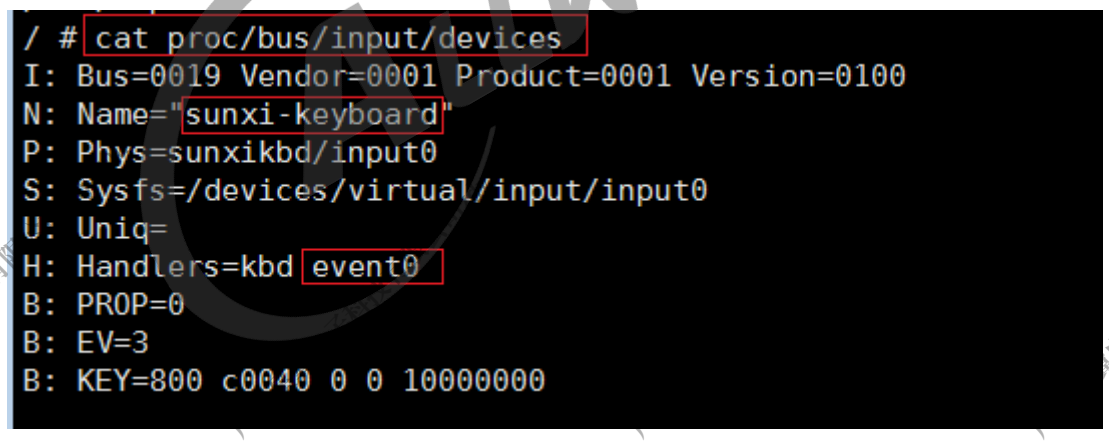

图 6-1: 查看 sunxi-keyboard 名称的 input 设备

6.2 基础调试手段

- 步骤 1

打开设备

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
/ # cd dev/input/
/dev/input # ls
event0  event1  event2  event3  mice    mouse0
```

图 6-2: 打开设备

- 步骤 2

根据 **6.1** 中查看的 Handlers，使用 hexdump 命令，接着按下按键，会报出类似以下数据。

```
/dev/input # hexdump event0
0000000 40d3 0000 e5d0 0007 0001 0066 0001 0000
0000010 40d3 0000 e5d0 0007 0000 0000 0000 0000
0000020 40d3 0000 c3cc 0009 0001 0066 0000 0000
0000030 40d3 0000 c3cc 0009 0000 0000 0000 0000
0000040 40d4 0000 4385 0004 0001 0066 0001 0000
0000050 40d4 0000 4385 0004 0000 0000 0000 0000
0000060 40d4 0000 2186 0006 0001 001c 0000 0000
0000070 40d4 0000 2186 0006 0000 0000 0000 0000
0000080 40d4 0000 beb2 000d 0001 008b 0001 0000
```

图 6-3: LRADC hexdump 数据

6.3 基础配置检查

- 步骤 1

SPI device tree 配置如下，需要检查和注意的是基地址、中断号。请查看芯片手册确认配置 **{中断号 =[芯片手册的中断号]-32}**。

```
keyboard0:keyboard{
    compatible = "allwinner,keyboard_1200mv";
    reg = <0x0 0x05070800 0x0 0x400>;
    interrupts = <GIC_SPI 50 IRQ_TYPE_NONE>;
    status = "okay";
    key_cnt = <5>;
    key0 = <190 115>;
    key1 = <390 114>;
    key2 = <600 139>;
    key3 = <800 28>;
    key4 = <980 102>;
};
```

每个 key 的第一个值表示各路采样电压值，可查看电路原理图进行更改，如下图红色框数值所示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

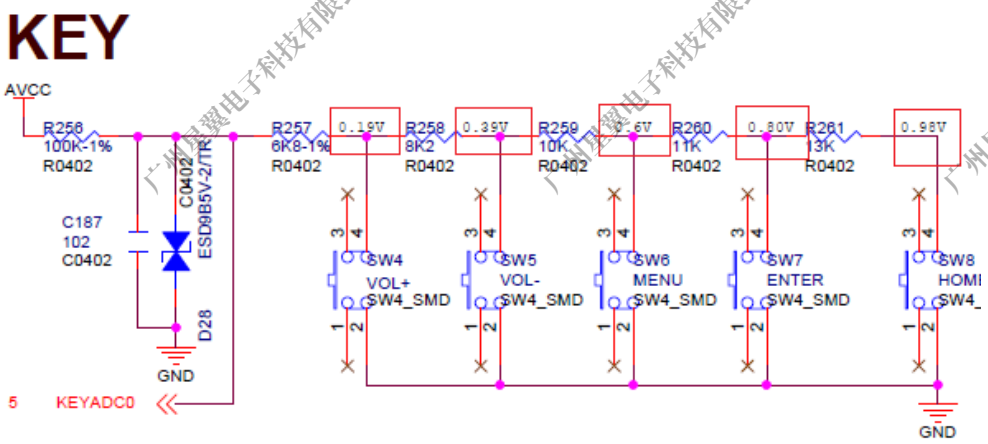

图 6-4: 电路原理图

- 步骤 2：menuconfig 的配置。

在命令行中进入内核 linux 目录，执行 make ARCH=arm menuconfig（64 位平台执行 make ARCH=arm64 menuconfig）进入配置主界面，并按以下步骤操作：

```
Device Drivers --->
 |----Input device support --->
 |   |----[*]Keyboards--->
 |   |   |----<*>softwinner KEY BOARD support
```

6.4 LRADC 常见问题排查

6.4.1 按键无反应

6.4.1.1 问题现象

在测试时，出现第一个按键或最后一个按键，按键不灵敏或没反应。

6.4.1.2 排查步骤

- 步骤 1

核查多块板子，硬件是否一样。按下每个按键时，用万用表测量如电路原理图处 KEYADC0 的电压值，是否和设备树每个 keyX 的第一个值一致，不一致就按测量值更改设备树的数值。

- 步骤 2

版权所有 © 珠海全志科技股份有限公司。保留一切权利

核查 AVCC 供电电压，是否为 1.2V，和设备树 compatible = “allwinner,keyboard_1200mv” 的属性值一致，不一致就更改设备树的属性值，allwinner,keyboard_1350mv 或 allwinner,keyboard_2000mv，分别调试。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 UART

7.1 串口打印正常，卡打印有问题

7.2 板上的某个串口异常

7.2.1 问题现象

在使用的过程中，串口打印可以正常的输入和输出，但卡打印只能输入不能输出。

7.2.2 排查步骤

- 步骤 1

检查硬件是否正常。使用万用表分别检查 rx 和 tx 是否有上拉的电压 3.3V，若无，可能硬件存在问题，或是管脚功能选择错误，进行下一步的排查。

- 步骤 2

利用 sunxi_dump 读取 port 的寄存器，对应芯片手册的值是否有误，有误请在 sys_config 里进行更改，注释掉使用相同这个引脚的模块。

7.3 串口上电第一次传输收到的数据是 0x00

7.3.1 问题现象

在使用奇偶校验时，上电仅第一次接收到的数据必定是 0x00。第二次及其之后正常。

7.3.2 排查步骤

- 步骤 1

版权所有 © 珠海全志科技股份有限公司。保留一切权利

检查 fifo 发现数据是正常的，打印寄存器发现使用的过程中，串口打印可以正常的输入和输出，但卡打印只能输入不能输出。LSR 寄存器（偏移值为 0x14）中 PE(bit3) 和 FE(bit4) 都为 error，uart 接收数据的代码里会对 LSR 寄存器做状态判断，发现 PE(Parity error) 和 FE(Framing error) 错误就将数据丢掉了。

- 步骤 2

和设计沟通，公司全平台都存在这个问题（设计猜测是 uart 的硬件代码最早做过修改，原先应该是用 DFF 搭的 FIFO 后面被换成了 memoryhardblock，前者缓存数据可以复位，后者复位不了，上电后 memory 值随机，导致误报中断），可以通过在 uart 第一次初始化时配置成 loopback 模式，自发自收，将第一个接收中断给冲掉，后续用户即可正常使用 uart 收发数据。

7.4 开启 S_UART 导致开机死机

7.4.1 问题现象

在使能 s_uart0 的情况下，系统开机死机，不能完全启动，串口完全无反应，同时 log 里也无任何报错或者警告。

7.4.2 排查步骤

- 步骤 1

检查硬件是否正常。使用万用表分别检查 rx 和 tx 是否有上拉的电压 3.3V，若无，可能硬件存在问题。

- 步骤 2

修改 env 环境信息，进入 boot console 控制台，在 console 控制台下配置 init_call 等级为 1：

```
env set initcall_debug 1;
env save
```

boot 启动查看有无报错或者警告出现，进行下一步的排查。

- 步骤 3

s_uart0 通常也被用作小 CPU 的 debug 信息输出端口，如果 kernel 也使用 s_uart，可能因为抢用资源而导致一个 CPU 挂掉。在 dts 中将 cpus_debug 部分 disable 掉，同时通过我司 FAE 找负责 CPUS 部分的同事沟通确认有无使用到 s_uart。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7.5 UART 无法正常发送数据

7.5.1 问题现象

在使用 uart 时无法发送数据。

7.5.2 排查步骤

- 步骤 1

通过查看 uart 的调试信息，确认 uart 是否正常发送数据。

- 步骤 2

查看 uart 的配置信息是否正确

```
cat /sys/bus/platform/devices/uart1/ctrl_info
```

- 步骤 3

查看 uart 的寄存器信息是否打开。

```
cat /sys/bus/platform/devices/uart1/status
```

- 步骤 4

查看 uart 的中断产生次数，确认是否有中断。如果没有 uart 中断产生，检查一下 uart 的 irq 号配置是否正确，控制器基地址是否正确。

```
cat /proc/interrupts
```

- 步骤 5

如果有中断产生，但是数据没有输出，检查 pin 的配置是否正确，pin 输出是否有拨码开关控制。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7.6 UART 串口接收数据异常

7.6.1 问题现象

在使用 uart 进行压力测试时，出现部分 uart 口接收数据丢失的现象，并且接收数据中出现多余的 0 字符。

7.6.2 排查步骤

- 步骤 1

如果使用两台机器进行互传测试，请先检查是否共地。

- 步骤 2

检查当前串口打印级别，是否级别太低导致中断性能问题（详细原因见本节原因描述）。尝试提高打印级别减少打印，重新测试问题是否解决。

- 步骤 3

检查当前所测试串口的 fifo 深度，当深度太小，会在中断响应不及时的情况下出现丢包问题（详细原因见本节原因描述）。尝试更换 fifo 更大的 uart 口重新测试，查看问题是否解决。

7.6.3 原因描述

在使用 uart 进行数据传输时，由于 printk 被大量调用（prink 执行会调用 local_irq_save 暂时关闭中断），出现了中断频繁被关闭的情况，导致 uart 中断到来不及时，无法从 uart fifo 中及时取出数据，这样，当数据继续涌向 rx 端，导致 fifo 溢出，出现”overrun“异常；同时，在驱动中出现 “overrun” 时会用 0 填充到 tty buf 中。

这样，就导致，主机端发送的数据被接收端接收时（用户层 read 时），出现了多出一个 0。并且，会出现接收数据丢失的情况。

7.7 UART 配置波特率与时钟不匹配

7.7.1 问题现象

在配置波特率时，波特率设置失败，导致数据接发异常。出错 log 示例：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[ 1022.369670] sw_uart_check_baudset()840 - uart2, select set 3, baud 460800, uartclk 24000000 beyond rance
        [30000000, 120000000]
```

7.7.2 排查步骤

- 步骤 1

需确认 sunxi uart 驱动 (tty/serial/sunxi-uart.c: sw_uart_check_baudset) 中是否支持对应波特率，出错会打印:

```
uart%d, baud %d beyond rance
```

- 步骤 2

需确认设置的波特率是否与 uart 时钟相匹配 (tty/serial/sunxi-uart.c: sw_uart_check_baudset)，如不匹配会打印错误。

```
"uart%d, select set %d, baud %d, uartclk %d beyond rance[%d, %d]
```

目前一般平台 uart 默认时钟为 24M，如果时钟不匹配，请看步骤三重新配置时钟

其中匹配关系摘录如下：

```
static struct baudset baud_set[] = {
  {115200, 24000000, 120000000},
  {230400, 30000000, 120000000},
  {380400, 24000000, 120000000},
  {460800, 30000000, 120000000},
  {921600, 30000000, 120000000},
  {1000000, 31000000, 120000000}, /* 31578947 */
  {1500000, 24000000, 120000000},
  {1750000, 54000000, 120000000}, /* 54545454 */
  {2000000, 31000000, 120000000}, /* 31578947 */
  {2500000, 40000000, 120000000}, /* 40000000 */
  {3000000, 46000000, 120000000}, /* 46153846 */
  {3250000, 54000000, 120000000}, /* 54545454 */
  {3500000, 54000000, 120000000}, /* 54545454 */
  {4000000, 66000000, 120000000}, /* 66666666 */
};
```

其中，baudset 数据结构定义如下：

```
struct baudset {
   u32 baud;
   u32 uartclk_min;
   u32 uartclk_max;
};
```

也就是说，baud_set 中，第一列为波特率；第二列为此波特率下，uart 时钟的最小时钟频率；第三列为此波特率下，uart 时钟的最大时钟频率。此数据结构用来判断用户层配置下来的波特率是否满足与当前 uart 时钟频率的关系。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

时钟与波特率关系：波特率 =uart_clk/(16*divisor) 其中 divisor 是 uart 内部的 16 位分频寄存器的值，因此 divisor 最大可以 65535。另外，我们需要注意波特率的精度，例如：24M 时钟，如果我们配置 9600 波特率，则 divisor 为 156，那我们实际得到的波特率为 9615，误差为 0.16%。不同外设对波特率有不同的精度需求

- 步骤 3

如果在步骤 2 中确认我们的时钟与波特率不匹配，则需要对 uart 时钟进行重新调整。这里举一个例子，如果我们现在需要配置波特率为 460800，发现 24M 时钟不满足条件，所以我们需要修改 uart 时钟频率。

一般只需要修改 dts，修改示例如下：

```
diff --git a/arch/arm64/boot/dts/sunxi/sun50iw9p1.dtsi b/arch/arm64/boot/dts/sunxi/sun50iw9p1.dtsi
index fe41ffab2af9..8affbb9d8923 100644
--- a/arch/arm64/boot/dts/sunxi/sun50iw9p1.dtsi
+++ b/arch/arm64/boot/dts/sunxi/sun50iw9p1.dtsi
@@ -673,7 +673,9 @@
                device_type = "uart2";
                reg = <0x0 0x05000800 0x0 0x400>;
                interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>;
-               clocks = <&clk_uart2>;
+               clocks = <&clk_uart2>, <&clk_apb2>, <&clk_psi>;
+               clock-frequency = <50000000>;
+               /* clocks = <&clk_uart2>; */
                pinctrl-names = "default", "sleep";
                pinctrl-0 = <&uart2_pins_a>;
                pinctrl-1 = <&uart2_pins_b>;
```

修改说明：

clocks 节点：

```
<模块时钟uart><总线时钟apb2><总线父时钟psi>
```

其中 uart 父时钟为 apb2（不可改变），apb2 总线父时钟为 psi（可选）。（**以前老旧芯片中，uart 父时钟可能挂载在 apb1 下。具体时钟关系应以芯片的 CCU 芯片手册为准**）

父时钟一般选择 psi 就可以满足要求。

clock-frequency 节点：

所需的 uart 时钟频率，也就是 apb2 输出的时钟频率。时钟选择应根据具体波特率选择（参考步骤 2），同时我们需要考虑此时钟频率能否从父时钟分频得到。这里，我们选择 50M，原因有两个：

1）可以直接由 200M 的 PSI 时钟 4 分频得到；

2）uart 可以计算得到较精确的波特率。

一般来说，apb2 默认父时钟为 dcxo24M，由于 apb2 时钟控制只有分频功能，所以，如果我们需要配置更高的频率，需要更换时钟源。在本例中，apb2 的父时钟选择 PSI（CCU_ 芯片手册查找）,PSI 一般为 200M（通过 clk debug 节点查看），之后通过 apb2 本身的分频就可以实现 50M。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

uart 本身没有自己的时钟控制，直接使用了 apb2 总线时钟，因此，如果我们要调整 uart 的时钟频率，也就是调整 apb2 的时钟频率。apb2 时钟可以分频控制（apb2_clk=parents/M/N），可以进行父时钟选择（OSC24M,RTC_32K,PSI,PLL_PERIO）。

注意

apb2 时钟同时是 twi 的时钟源，因此，对 apb2 时钟的修改会影响 twi 的时钟。

注意

由于所有 uart 共用 apb2 时钟源，因此，对某个 uart 口配置了其他时钟源后，需要注意可能会影响到其他 uart 口的波特率（例如常用的 uart0 串口打印出现乱码）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8 SMART CARD

8.1 SCR 无法获取 ATR 数据

8.1.1 问题现象

在使用某些厂家的电视卡时，无法获取 ATR 应答数据。

8.1.2 排查步骤

- 步骤 1

通过打开 SCR 模块的调试信息，发现 ATR 获取失败了，但是可以看到是有 RX 中断进来的。

- 步骤 2

通过时间戳发现 RX 中断是在 GET ATR 失败之后才到来。

- 步骤 3

在 1M clock 的情况下，使用示波器测量，发现在 reset 之后 ATR 需要 73ms 之后才会到来，根据 ISO7816 协议里规定 ATR 的标准时间在 400clock < ATR time < 40000clock，在 1M clock 时，也就是最大不会超过 40ms，可见这张卡是不合标准的。

- 步骤 4

我们 SOC 里获取 ATR 时间的寄存器预留是 16 个 bit，也就是 65535，在 1M clock 时钟下，最大支持 65ms 左右，明显不支持 73ms 的电视卡，所以不支持硬件读取 ATR，只能软件来读取。实际上，在 GET ATR Failed 之后，还是有 ATR 数据进来的，只要及时读取，也能获取到 ATR 数据，之后进行解码即可。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。